Universidad San Carlos de Guatemala

Centro Universitario de Occidente



Curso: Seminario de Sistemas 1

Catedrático: Ing. Mario Tobar

Auxiliar: Luis Tax

Proyecto: FlorArte

Manual Técnico

Nombre: Marco Pool Chiché Sapón

Carné: 201832053

Quetzaltenango, octubre de 2026

Índice

Presentación....3
1. Resumen ejecutivo y alcance....4
2. Arquitectura del sistema....8
3. Modelo de datos PostgreSQL....13
4. Catálogo de API REST....27
5. Instalación y despliegue....63
6. Seguridad, pruebas y CI/CD....66
7. Matriz de trazabilidad....71
8. Repositorio y código fuente....74
9. Integración continua y entrega de imágenes....76
9.1. Flujo de integración....76
9.2. Ejecuciones de GitHub Actions....76
9.3. Registro GHCR y distribución....77
10. Despliegue y operación....78
10.1. Arquitectura de producción....78
10.2. Dominio y HTTPS....78
10.3. Contenedores y actualización manual....79
10.4. Respaldo y restauración de datos....80
10.5. Mantenimiento....80

Presentación

Este manual describe la arquitectura, el modelo de datos, los contratos de la API, la instalación, la seguridad y la operación de FlorArte. Su propósito es facilitar la comprensión y el mantenimiento del sistema desarrollado para el curso Seminario de Sistemas 1.

La aplicación integra un frontend React y un backend Spring Boot con PostgreSQL. Los capítulos conservan el detalle técnico de entidades, relaciones, validaciones y servicios, y lo relacionan con los módulos operativos de la floristería.

Las ilustraciones de operación utilizan datos ficticios. Los diagramas representan los componentes del sistema y sus responsabilidades. La integración continua y el despliegue manual se describen como procesos diferentes.

1. Resumen ejecutivo y alcance

Finalidad y problema atendido

FlorArte centraliza el catálogo de flores, las entradas de inventario, las mermas, los pedidos, los arreglos y los eventos. Su finalidad es organizar la información de la floristería y facilitar el seguimiento de existencias y operaciones.

El alcance comprende personas y cuentas, control de entradas y salidas, seguimiento de pedidos y eventos, conservación de comprobantes, consulta de indicadores y control de acceso por rol.

Alcance funcional real

Capacidad	Descripción
Landing pública	Flor_Arte_Fronted/src/pages/Landing.tsx, Flor_Arte_Fronted/src/styles/landing.module.css, Flor_Arte_Fronted/src/routes/AppRoutes.tsx: / presenta empresa, flores/arreglos/eventos, contacto e ingreso /login. Usa fotografías externas de referencia y logo local. Los enlaces de contacto no crean pedidos en la base.
Login y acceso	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java, Flor_Arte_Fronted/src/components/login/LoginForm.tsx: correo/contraseña, JWT y desafío opcional 2FA.
2FA y recuperación	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/DesafioAuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/RecuperacionService.java, V18: configuración individual, códigos por correo y enlaces de un uso.
Personas	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PersonaService.java, Flor_Arte_Fronted/src/pages/shared/personas/Formulario.tsx: nombre/rol requeridos; DPI/contactos opcionales y normalizados.
Usuarios	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/UserService.java: alta por administrador de cuenta asociada a persona ADMINISTRADOR/EMPLEADO. No hay registro público. Algunas consultas permiten EMPLEADO; ver 06.
Catálogo floral	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java: tipos, colores, combinación única tipo/color, precio, stock y disponibilidad. La imagen de la flor procede del tipo.

Capacidad	Descripción
Compras e inventario	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Entra_InvService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Deta_EntraService.java: entradas con detalle, costos, subtotales y efectos en stock.
Merma	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Mov_InvService.java, V16: salida de flores, costo unitario histórico y pérdida estimada. Registros antiguos pueden carecer de costo.
Pedidos de flores	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PedidoService.java, Flor_Arte_Fronted/src/pages/shared/pedidos/Create.tsx: cliente, responsable, detalles, cantidades/precios, fecha, total y estado. Auditoría previa detectó truncamiento de decimales; no se corrige aquí.
Arreglos y sus pedidos	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ArregloService.java, V11–V12: receta de flores, imagen URL, capacidad calculada; pedido guarda nombre/precio/composición de venta. No hay inventario físico independiente de arreglos.
Eventos	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/EventoService.java, V13/V15: cliente, fecha, preparación, flores, reserva al confirmar, estados e importe/pago. El estado no cambia automáticamente por llegar la fecha.
Recibos/comprobantes	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobantePdfService.java, V17: snapshot JSONB de pedido/arreglo/evento/compra; PDF y envío con adjunto. No equivale a factura fiscal ni confirma por sí solo el pago.
Reportes	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java, V14–V16, Flor_Arte_Fronted/src/pages/shared/reportes/Index.tsx: ingresos, compras, merma, rankings y filtros día/semana/mes/año. Solo ADMINISTRADOR.
Exportación PDF/Excel	Flor_Arte_Fronted/src/services/exportarDocumentos.ts: usa JSON de reportes y logo; PDF de comprobantes generado en backend. No asumir endpoints binarios de exportación de reportes.
Inicio y avisos	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java, Flor_Arte_Fronted/src/hooks/useResumenInicio.ts, headers admin/empleado: contadores, agenda y avisos consultados por HTTP. No hay WebSocket ni notificación push comprobada.
Portal CLIENTE/PROVEEDOR	Ambos tipos existen en RoleList/persona, pero no hay rutas/paneles propios ni alta normal de esas cuentas.

Capacidad	Descripción
	Login de rol ajeno deriva a /.

Usuarios y alcance organizacional

ADMINISTRADOR y EMPLEADO son usuarios operativos del panel. CLIENTE y PROVEEDOR clasifican contactos para pedidos y compras. Cada persona tiene un rol; cada usuario referencia una persona de forma única. No se implementa una relación de múltiples roles por usuario. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/enums/RoleList.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Rol.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Persona.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/User.java, Flor_Arte_Backend/src/main/resources/db/migration/V1__crear_rol.sql, Flor_Arte_Backend/src/main/resources/db/migration/V2__crear_persona.sql, Flor_Arte_Backend/src/main/resources/db/migration/V3__crear_usuario.sql.

El empleado comparte módulos operativos con el administrador, con un plazo de edición/eliminación de 30 minutos en los registros definidos en ReglasEdicion; la modificación de estado tiene rutas separadas sin ese plazo. El permiso depende de la creación del registro, no de quién lo creó. Los catálogos, personas, detalles y usuarios tienen políticas específicas: no generalizar el plazo a todo el sistema. Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReglasEdicion.java, inventario completo 04.

Tecnologías declaradas en las fuentes

Capa	Tecnología/version declarada	Evidencia
Frontend	React ^19.2.8, TypeScript ~6.0.2, Vite ^8.2.0	Flor_Arte_Fronted/package.json; los rangos no prueban la versión desplegada.
Navegación/HTTP	React Router ^7.18.2, Axios ^1.19.0	Flor_Arte_Fronted/package.json, Flor_Arte_Fronted/src/App.tsx, Flor_Arte_Fronted/src/services/apiService.ts.
Presentación	Bootstrap ^5.3.8, Bootstrap Icons ^1.13.1; CSS propio y fuentes locales	Flor_Arte_Fronted/src/main.tsx, Flor_Arte_Fronted/src/styles/fuentes.css.
Documentos cliente	jsPDF ^4.2.1, AutoTable ^5.0.8, ExcelJS ^4.4.0	Flor_Arte_Fronted/package.json, Flor_Arte_Fronted/src/services/exportarDocumentos.ts.
Backend	Java 21, Spring Boot 4.1.0, Maven Wrapper	Flor_Arte_Backend/ pom.xml, .mvn/wrapper/maven-wrapper.properties.
Persistencia	Spring Data JPA/Hibernate + JdbcTemplate y NamedParameterJdbcTemplate	pom y servicios; arquitectura híbrida.
Seguridad	Spring Security, BCrypt, JJWT 0.12.6, HMAC-SHA256/SHA-256	pom, Flor_Arte_Backend/src/main/java/co

Capa	Tecnología/version declarada	Evidencia
		m/florarte/backend/jwt/JwtUtil.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/SecretosAuth.java.
Base de datos	PostgreSQL, Flyway, 18 migraciones, 21 tablas de negocio y 10 vistas	Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql y documento 03.
Correo/PDF servidor	JavaMailSender, Gmail SMTP STARTTLS, PDFBox 3.0.8	pom, Flor_Arte_Backend/src/main/java/com/florarte/backend/config/CorreoConfig.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobantePdfService.java.
Pruebas	JUnit/Spring Boot Test/Mockito y scripts Playwright	Flor_Arte_Backend/src/test, Flor_Arte_Fronted/tests, documento 06.

Alcance del producto

El sistema administra operaciones internas de la floristería. El calendario organiza fechas de eventos y preparación; las reservas se aplican a las cantidades de flores. La emisión de comprobantes es de carácter operativo y no equivale a facturación electrónica FEL ni a contabilidad integral.

2. Arquitectura del sistema

Vista lógica comprobada

FlorArte separa la interfaz React de la API Spring Boot y utiliza PostgreSQL para la persistencia. Axios intercambia solicitudes REST en formato JSON. El backend organiza controladores, servicios y acceso a datos; Spring Security controla la autenticación y autorización.

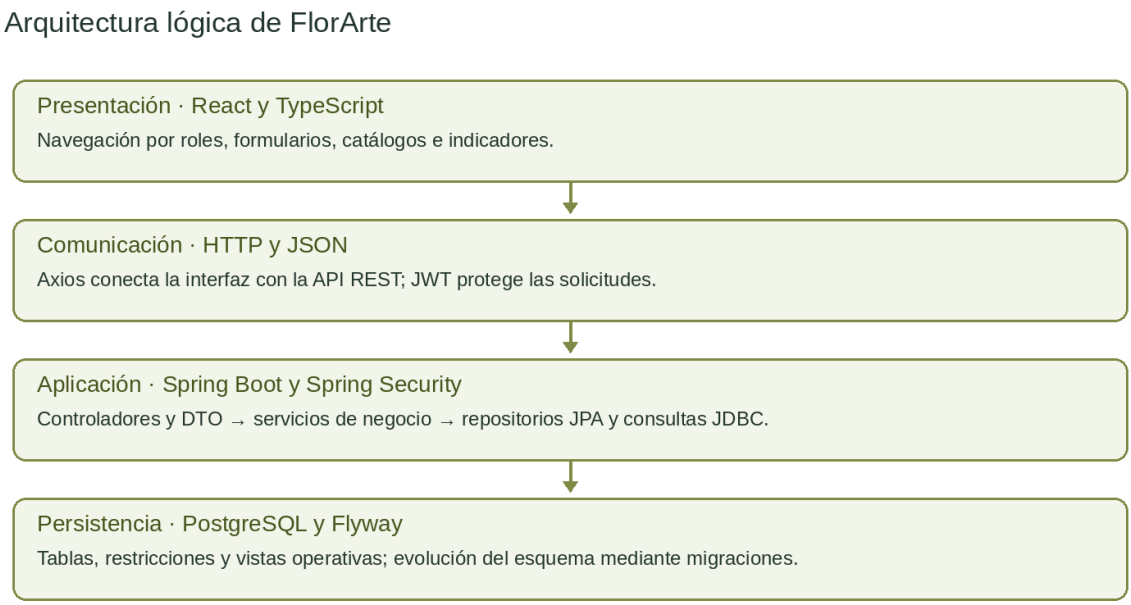


Figura T1. Arquitectura lógica y responsabilidades de FlorArte.

Elaboración propia a partir de los componentes de presentación, servicios y persistencia.

Estructura y responsabilidades

Área	Responsabilidad	Fuentes
Arranque SPA	StrictMode, BrowserRouter, rutas y CSS global	Flor_Arte_Fronted/src/main.tsx, Flor_Arte_Fronted/src/App.tsx.
Rutas públicas	Landing /, login, recuperación y restablecimiento	Flor_Arte_Fronted/src/routes/ AppRoutes.tsx.
Rutas por rol	/admin y /empleado con layouts y PrivateRoute	Flor_Arte_Fronted/src/routes/ AdminRoutes.tsx, Flor_Arte_Fronted/src/routes/EmpleadoRoute.tsx, Flor_Arte_Fronted/src/components/PrivateRoute.tsx.
Gestión común	Flores, compras/merma, pedidos, arreglos/eventos montados bajo ambos paneles	Flor_Arte_Fronted/src/routes/ GestionRoutes.tsx.
Componentes reutilizados	Listado, detalle, selector, calendario de mes, modal, campo numérico,	Flor_Arte_Fronted/src/components/ gestion/Listado.tsx,

Área	Responsabilidad	Fuentes
	permisos	Flor_Arte_Fronted/src/components/shared/SelectorRegistro.tsx, Flor_Arte_Fronted/src/components/shared/Modal.tsx, Flor_Arte_Fronted/src/components/shared/CampoNumero.tsx.
Estado cliente	useState/useEffect/hooks; no Redux ni store global identificado	Flor_Arte_Fronted/src/hooks/usePagina.ts, Flor_Arte_Fronted/src/hooks/useResumenInicio.ts.
HTTP	Cliente público sin JWT y privado con interceptor Bearer; URL obligatoria	Flor_Arte_Fronted/src/services/apiService.ts, Flor_Arte_Fronted/src/config/api.ts.
Entrada API	Controllers + validación Jakarta; retorno ResponseEntity/Map	Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java.
Negocio	Transacciones, cálculo financiero, stock, permisos y flujos	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PedidoService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ArregloService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/EventoService.java.
Persistencia base	Entity/Repository para personas, usuarios, flores, inventario y pedidos	Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/User.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/repositories/PedidoRepository.java.
Persistencia SQL	Arreglos/eventos, seguridad por correo, comprobantes y reportes usan JDBC	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ArregloService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/DesafioAuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReporteService.java.
Fallos	Advice global, excepciones de flujo, EntryPoint y formato propio de cuenta inactiva	Flor_Arte_Backend/src/main/java/com/florarte/backend/exceptions/GlobalExceptionHandler.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtEntryPoint.java.

Autenticación, autorización y sesión

El estado de sesión HTTP en Spring Security es STATELESS. El JWT contiene subject/correo, role, authVersion, emisión y expiración; jwt.expiration es en segundos (3600 en configuración local inspeccionada, valor sustituible). La firma se calcula con una clave HMAC del servidor mediante JJWT; el algoritmo concreto se selecciona al firmar según la clave. El algoritmo de firma depende de la clave y de la selección efectuada por JJWT. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/config/SecurityConfig.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtUtil.java.

En cada petición protegida, el filtro valida firma/expiración, carga usuario activo y autoridades actuales y compara authVersion con la base. Un usuario desactivado no establece una sesión válida. El login no escribe SecurityContext antes del segundo factor. El control de rutas de React solo mejora navegación: no sustituye al backend. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtAuthenticationFilter.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/AuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/UserService.java.

React guarda token, role y user en localStorage al completar el login. Desafío y OTP son estado en memoria; no se guardan allí contraseñas ni enlaces de recuperación. Interceptor elimina sesión local y redirige al login ante 401 de módulos. Logout elimina datos locales; no hay endpoint de logout que revoque inmediatamente un JWT copiado. La sesión utiliza el JWT emitido al autenticar la cuenta. Fuentes: Flor_Arte_Fronted/src/services/authService.ts, Flor_Arte_Fronted/src/services/apiService.ts, Flor_Arte_Fronted/src/pages/shared/perfil/Perfil.tsx.

Etapa	Proceso de autenticación
Credenciales	React envía el correo y contraseña a AuthController.
Validación	El servicio valida la cuenta, su estado y el segundo factor configurado.
Segundo factor	La cuenta con 2FA recibe un desafío temporal y verifica el código de correo.
Sesión	La validación correcta entrega el JWT y el rol de acceso.

Flujo de recuperación

La recuperación genera un enlace temporal de un solo uso. El navegador retira su token de la dirección visible y el backend valida su hash, vigencia y versión de cuenta antes de actualizar la contraseña e invalidar los accesos anteriores.

Flujos comerciales y consistencia

Compra: cabecera/detalles → total y aumento de stock → movimientos. Merma: valida cantidad/costo y descuenta stock. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Entra_InvService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Deta_EntraService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Mov_InvService.java.

Pedido floral: suma cantidad × precio de cada detalle, descuenta existencias y registra venta; cancelar repone, reactivar requiere stock. Actualización completa reconcilia detalles y stock en

transacción. Fuentes:
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PedidoService.java,
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Deta_PediService.java.

Arreglo: catálogo contiene receta; capacidad = mínimo de stock/cantidad requerida por flor, con flor no disponible contando cero. Crear receta no consume; vender sí. Pedido conserva su composición y precio históricos. Fuente:
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ArregloService.java.

Evento: por atender conserva necesidades; confirmar reserva descontando stock; cancelar desde estados permitidos repone. No se espera a una proximidad automática de fecha para reservar. Cambio a TRABAJANDO requiere día del evento o posterior. Realizado/pagado conserva composición. Fuente:
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/EventoService.java.

Comprobante: copia JSONB de operación, tipo/origen únicos; PDF desde snapshot y correo con adjunto. No confirma un cobro por sí mismo. Fuentes:
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteService.java,
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteCorreoService.java, V17.

Los servicios utilizan transacciones. En los procesos JDBC de arreglos, eventos y autenticación se emplean bloqueos de fila FOR UPDATE. JPA y JDBC comparten el origen de datos configurado para la aplicación.

Reportes, paginación y avisos

Listados consultan página/tamaño, búsqueda y filtros en servidor; se cancelan consultas anteriores con AbortController. La búsqueda tiene retraso corto para evitar una petición por tecla. resumen agrega todo el filtro, no solo la página. Fuente:
Flor_Arte_Fronted/src/hooks/usePagina.ts,
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java.

Reportes leen vistas SQL y agrupan rankings; exportaciones de reportes se generan en el navegador con dependencias cargadas dinámicamente. El balance es ingresos menos compras y muestra la merma por separado. Fuentes:
Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReporteService.java,
Flor_Arte_Fronted/src/services/exportarDocumentos.ts.

Los avisos son consultas de stock bajo, pedidos por atender, eventos y, según rol, cuentas/responsables. Headers hacen polling y guardan IDs leídos en localStorage. No existe una tabla de notificaciones persistentes ni servidor push identificado. Fuentes:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java,
Flor_Arte_Fronted/src/components/admin/Header.tsx,
Flor_Arte_Fronted/src/components/empleado/Header.tsx.

Decisiones de arquitectura

La aplicación utiliza un backend modular por capas y una base relacional compartida. El control visual de rutas complementa la autorización del servidor. La instalación descrita utiliza contenedores y un proxy HTTPS, como se explica en el capítulo 10.

3. Modelo de datos PostgreSQL

Alcance y autoridad del modelo

El esquema de la aplicación se define mediante las migraciones Flyway V1–V18. El archivo DataBase/script.sql crea la base; las tablas, vistas, índices y restricciones se administran en las migraciones versionadas. El diccionario siguiente describe ese modelo.

SERIAL/BIGSERIAL generan identificadores con secuencia; no son UUID. Las tablas de desafíos usan UUID por otra necesidad. Se distingue TIMESTAMP comercial sin zona de TIMESTAMPTZ para instantes de creación/autenticación. La aplicación usa America/Guatemala para varias fechas comerciales y UTC para seguridad; comprobar coherencia de JVM/servidor antes del despliegue.

Diccionario de tablas (estado posterior a V18)

Las columnas PK son implícitamente NOT NULL aunque el texto DDL no lo repita. La obligatoriedad indicada aquí corresponde a SQL; las pantallas y DTO pueden imponer más condiciones. Las FK no prueban por sí mismas el rol de la persona referenciada: esa validación se realiza en servicios.

rol

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V1__crear_rol.sql.

Campo	Tipo y reglas SQL	Evidencia
id_rol	BIGSERIAL PRIMARY KEY	V1
tipo	VARCHAR(50) NOT NULL UNIQUE	V1

persona

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V2__crear_persona.sql.

Campo	Tipo y reglas SQL	Evidencia
id_persona	BIGSERIAL PRIMARY KEY	V2
nombre	VARCHAR(100) NOT NULL	V2
telefono	VARCHAR(20)	V2
dpi	VARCHAR(20) UNIQUE	V2
correo	VARCHAR(150) UNIQUE	V2
id_rol	BIGINT NOT NULL	V2

Restricciones de tabla:

CONSTRAINT fk_persona_rol FOREIGN KEY (id_rol) REFERENCES rol(id_rol).

usuario

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V3__crear_usuario.sql.

Campo	Tipo y reglas SQL	Evidencia
id_usuario	BIGSERIAL PRIMARY KEY	V3

Campo	Tipo y reglas SQL	Evidencia
name	VARCHAR(100) NOT NULL UNIQUE	V3
email	VARCHAR(150) NOT NULL UNIQUE	V3
password	VARCHAR(255) NOT NULL	V3
token	VARCHAR(500)	V3
estado	BOOLEAN NOT NULL DEFAULT TRUE	V3
id_persona	BIGINT NOT NULL UNIQUE	V3
twofa_habilitado	BOOLEAN NOT NULL DEFAULT FALSE	V18
auth_version	BIGINT NOT NULL DEFAULT 0 CHECK (auth_version>=0)	V18

Restricciones de tabla:

CONSTRAINT fk_usuario_persona FOREIGN KEY (id_persona) REFERENCES persona(id_persona).

color

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V5__modulo_flores.sql.

Campo	Tipo y reglas SQL	Evidencia
id_color	SERIAL PRIMARY KEY	V5
nombre	VARCHAR(50) NOT NULL UNIQUE	V5

tipo_flor

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V5__modulo_flores.sql.

Campo	Tipo y reglas SQL	Evidencia
id_tipo_flor	SERIAL PRIMARY KEY	V5
nombre	VARCHAR(100) NOT NULL UNIQUE	V5
descripcion	TEXT	V5
imagen_url	TEXT	V5

flor

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V5__modulo_flores.sql.

Campo	Tipo y reglas SQL	Evidencia
id_flor	SERIAL PRIMARY KEY	V5
id_tipo_flor	INT NOT NULL	V5
id_color	INT NOT NULL	V5
precio	NUMERIC(10,2) NOT NULL CHECK (precio >= 0)	V5
stock	INT NOT NULL DEFAULT 0 CHECK (stock >= 0)	V5
estado	BOOLEAN NOT NULL DEFAULT	V5

Campo	Tipo y reglas SQL	Evidencia
	TRUE	
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP (V9)	V9

Restricciones de tabla:

CONSTRAINT fk_flor_tipo FOREIGN KEY (id_tipo_flor) REFERENCES tipo_flor(id_tipo_flor).

CONSTRAINT fk_flor_color FOREIGN KEY (id_color) REFERENCES color(id_color).

CONSTRAINT uq_flor_tipo_color UNIQUE (id_tipo_flor, id_color).

entrada_inventario

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V6__modulo_inventario.sql.

Campo	Tipo y reglas SQL	Evidencia
id_entrada	SERIAL PRIMARY KEY	V6
id_persona	BIGINT NOT NULL	V6
fecha	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V6
total	NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total >= 0)	V6
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP (V9)	V9

Restricciones de tabla:

CONSTRAINT fk_entrada_proveedor FOREIGN KEY (id_persona) REFERENCES persona(id_persona).

detalle_entrada

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V6__modulo_inventario.sql.

Campo	Tipo y reglas SQL	Evidencia
id_detalle_entrada	SERIAL PRIMARY KEY	V6
id_entrada	INT NOT NULL	V6
id_flor	INT NOT NULL	V6
cantidad	INT NOT NULL CHECK (cantidad > 0)	V6
precio_compra	NUMERIC(10,2) NOT NULL CHECK (precio_compra >= 0)	V6
subtotal	NUMERIC(10,2) NOT NULL CHECK (subtotal >= 0)	V6

Restricciones de tabla:

CONSTRAINT fk_detalle_entrada FOREIGN KEY (id_entrada) REFERENCES entrada_inventario(id_entrada) ON DELETE CASCADE.

CONSTRAINT fk_detalle_entrada_flor FOREIGN KEY (id_flor) REFERENCES flor(id_flor).

movimiento_inventario

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V6__modulo_inventario.sql.

Campo	Tipo y reglas SQL	Evidencia
id_movimiento_inventario	SERIAL PRIMARY KEY	V6
id_flor	INT NOT NULL	V6
tipo_movimiento	VARCHAR(20) NOT NULL	V6
cantidad	INT NOT NULL CHECK (cantidad > 0)	V6
motivo	VARCHAR(50) NOT NULL	V6
fecha	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V6
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP (V9)	V9
costo_unitario	NUMERIC(16,6) CHECK (costo_unitario>=0)	V16

Restricciones de tabla:

CONSTRAINT fk_movimiento_flor FOREIGN KEY (id_flor) REFERENCES flor(id_flor).

CONSTRAINT chk_tipo_movimiento CHECK (tipo_movimiento IN ('ENTRADA', 'SALIDA')).

CONSTRAINT chk_motivo_movimiento CHECK (motivo IN ('ENTRADA', 'VENTA', 'ARREGLO', 'MERMA')).

pedido

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V7__modulo_pedidos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_pedido	SERIAL PRIMARY KEY	V7
id_cliente	BIGINT NOT NULL	V7
id_empleado	BIGINT	V7
estado	VARCHAR(30) NOT NULL DEFAULT 'PENDIENTE'	V7
total	NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total >= 0)	V7
fecha	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V8
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP (V9)	V9

Restricciones de tabla:

CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente) REFERENCES persona(id_persona).

CONSTRAINT fk_pedido_empleado FOREIGN KEY (id_empleado) REFERENCES persona(id_persona).

CONSTRAINT chk_estado_pedido CHECK (estado IN ('PENDIENTE', 'CONFIRMADO', 'PREPARANDO', 'LISTO', 'ENTREGADO', 'CANCELADO')).

detalle_pedido

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V7__modulo_pedidos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_detalle_pedido	SERIAL PRIMARY KEY	V7
id_pedido	INT NOT NULL	V7
id_flor	INT NOT NULL	V7
cantidad	INT NOT NULL CHECK (cantidad > 0)	V7
precio	NUMERIC(10,2) NOT NULL CHECK (precio >= 0)	V7
subtotal	NUMERIC(10,2) NOT NULL CHECK (subtotal >= 0)	V7

Restricciones de tabla:

CONSTRAINT fk_detalle_pedido FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido) ON DELETE CASCADE.

CONSTRAINT fk_detalle_pedido_flor FOREIGN KEY (id_flor) REFERENCES flor(id_flor).

arreglo

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V11__modulo_arreglos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_arreglo	SERIAL PRIMARY KEY	V11
nombre	VARCHAR(100) NOT NULL UNIQUE CHECK (btrim(nombre) <> '')	V11
descripcion	TEXT	V11
precio	NUMERIC(10,2) NOT NULL CHECK (precio >= 0)	V11
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V11
imagen_url	VARCHAR(2048)	V12

detalle_arreglo

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V11__modulo_arreglos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_detalle_arreglo	SERIAL PRIMARY KEY	V11
id_arreglo	INT NOT NULL REFERENCES arreglo(id_arreglo) ON DELETE CASCADE	V11
id_flor	INT NOT NULL REFERENCES flor(id_flor)	V11
cantidad	INT NOT NULL CHECK (cantidad > 0)	V11

Restricciones de tabla:

UNIQUE (id_arreglo,id_flor).

pedido_arreglo

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V11__modulo_arreglos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_pedido_arreglo	SERIAL PRIMARY KEY	V11
id_cliente	BIGINT NOT NULL REFERENCES persona(id_persona)	V11
id_empleado	BIGINT REFERENCES persona(id_persona)	V11
id_arreglo	INT NOT NULL REFERENCES arreglo(id_arreglo)	V11
nombre_arreglo	VARCHAR(100) NOT NULL	V11
cantidad	INT NOT NULL DEFAULT 1 CHECK (cantidad > 0)	V11
precio_unitario	NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0)	V11
total	NUMERIC(10,2) NOT NULL CHECK (total >= 0)	V11
fecha	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V11
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V11
estado	VARCHAR(30) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE','CONFIRMADO','PREPARANDO','LISTO','ENTREGADO','CANCELADO'))	V11

detalle_pedido_arreglo

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V11__modulo_arreglos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_detalle_pedido_arreglo	SERIAL PRIMARY KEY	V11
id_pedido_arreglo	INT NOT NULL REFERENCES pedido_arreglo(id_pedido_arreglo) ON DELETE CASCADE	V11
id_flor	INT NOT NULL REFERENCES flor(id_flor)	V11
cantidad	INT NOT NULL CHECK (cantidad > 0)	V11

Restricciones de tabla:

UNIQUE (id_pedido_arreglo,id_flor).

evento

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V13__modulo_eventos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_evento	SERIAL PRIMARY KEY	V13
id_cliente	BIGINT NOT NULL REFERENCES persona(id_persona)	V13
id_empleado	BIGINT REFERENCES persona(id_persona)	V13
nombre	VARCHAR(150) NOT NULL CHECK (btrim(nombre) <> '')	V13
fecha	TIMESTAMP NOT NULL	V13
dias_preparacion	INT NOT NULL DEFAULT 0 CHECK (dias_preparacion BETWEEN 0 AND 365)	V13
descripcion	TEXT	V13
ubicacion	VARCHAR(250)	V13
estado	VARCHAR(30) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE','CONFIRMADO','TRABAJANDO','REALIZADO','PAGADO','CANCELADO'))	V13
reserva_aplicada	BOOLEAN NOT NULL DEFAULT FALSE	V13
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V13
importe	NUMERIC(10,2) CHECK (importe>=0)	V15
fecha_pago	TIMESTAMP	V15

Restricciones de tabla:

CONSTRAINT chk_evento_reserva CHECK (reserva_aplicada = (estado IN ('CONFIRMADO','TRABAJANDO','REALIZADO','PAGADO'))).

detalle_evento

Definición: Flor_Arte_Backend/src/main/resources/db/migration/V13__modulo_eventos.sql.

Campo	Tipo y reglas SQL	Evidencia
id_detalle_evento	SERIAL PRIMARY KEY	V13
id_evento	INT NOT NULL REFERENCES evento(id_evento) ON DELETE CASCADE	V13
id_flor	INT NOT NULL REFERENCES flor(id_flor)	V13
cantidad	INT NOT NULL CHECK (cantidad > 0)	V13

Restricciones de tabla:

UNIQUE (id_evento,id_flor).

comprobante

Definición:
Flor_Arte_Backend/src/main/resources/db/migration/V17__comprobantes_operaciones.sql.

Campo	Tipo y reglas SQL	Evidencia
id_comprobante	BIGSERIAL PRIMARY KEY	V17
tipo	VARCHAR(20) NOT NULL CHECK (tipo IN ('PEDIDO','ARREGLO','EVENTO','INVENTARIO'))	V17
id_origen	INT NOT NULL CHECK (id_origen>0)	V17
documento	JSONB NOT NULL CHECK (jsonb_typeof(documento)='object')	V17
emitido_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V17
emitido_por	VARCHAR(100) NOT NULL	V17

Restricciones de tabla:

UNIQUE(tipo,id_origen).

desafio_2fa

Definición:
Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql.

Campo	Tipo y reglas SQL	Evidencia
id_desafio	UUID PRIMARY KEY	V18
id_usuario	BIGINT NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE	V18
proposito	VARCHAR(15) NOT NULL CHECK (proposito IN ('LOGIN','CONFIGURAR'))	V18
habilitar	BOOLEAN	V18
codigo_hash	CHAR(64) NOT NULL	V18
auth_version	BIGINT NOT NULL	V18
expira_en	TIMESTAMPTZ NOT NULL	V18
limite_en	TIMESTAMPTZ NOT NULL	V18
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V18
reenviado_en	TIMESTAMPTZ NOT NULL	V18
intentos	INT NOT NULL DEFAULT 0 CHECK (intentos BETWEEN 0 AND 5)	V18
reenvios	INT NOT NULL DEFAULT 0 CHECK (reenvios BETWEEN 0 AND 3)	V18
utilizado	BOOLEAN NOT NULL DEFAULT	V18

Campo	Tipo y reglas SQL	Evidencia
	FALSE	

Restricciones de tabla:

CHECK ((proposito='CONFIGURAR')=(habilitar IS NOT NULL)).

recuperacion_password

Definición:
Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql.

Campo	Tipo y reglas SQL	Evidencia
token_hash	CHAR(64) PRIMARY KEY	V18
id_usuario	BIGINT NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE	V18
auth_version	BIGINT NOT NULL	V18
expira_en	TIMESTAMPTZ NOT NULL	V18
creado_en	TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP	V18
utilizado	BOOLEAN NOT NULL DEFAULT FALSE	V18

limite_autenticacion

Definición:
Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql.

Campo	Tipo y reglas SQL	Evidencia
clave	CHAR(64) PRIMARY KEY	V18
inicio_en	TIMESTAMPTZ NOT NULL	V18
cantidad	INT NOT NULL CHECK (cantidad>0)	V18

Modificaciones y reglas que complementan las tablas

V9: creado_en es obligatorio con valor por defecto en flor, entrada, pedido y movimiento. Los registros previos se fechan 31 minutos antes para no reabrir la edición de EMPLEADO. No cambia al modificar la fecha comercial.

V10: cadenas vacías de correo/DPI/teléfono en persona se convierten a NULL. La unicidad de PostgreSQL permite varios NULL; dos personas sin DPI no deben competir por una cadena vacía. Normalización de entradas nuevas: PersonaService.

V12: agrega arreglo.imagen_url VARCHAR(2048); no hay tabla de archivos ni almacenamiento S3 verificable.

V13: el CHECK vigente de motivos de movimiento incluye EVENTO, además de ENTRADA/VENTA/ARREGLO/MERMA. No utilizar solo el CHECK original de V6.

V15: agrega importe y fecha de pago del evento, ambos originalmente opcionales para preservar históricos desconocidos. El servicio exige importe al pagar.

V16: costo de merma NUMERIC(16,6), opcional en SQL por compatibilidad histórica; nuevas mermas necesitan costo recibido o calculable por compras.

V18: el trigger trg_revocar_accesos_usuario ejecuta revocar_accesos_usuario() antes de actualizar usuario. Cambios de contraseña, email, estado, persona o configuración 2FA incrementan auth_version, limpian el token heredado e invalidan desafíos y recuperaciones. El cambio del rol en la tabla persona no forma parte de ese trigger; el filtro recarga el rol vigente para autorizar.

comprobante.id_origen no es una FK a una tabla concreta: es referencia lógica discriminada por tipo. El JSONB guarda una copia de empresa, persona, responsable, fecha, estado, total y líneas. Se conserva al cambiar/eliminar la operación, pero el propio comprobante sí tiene endpoint de eliminación con permisos. UNIQUE(tipo,id_origen) evita emitir duplicados mientras exista el documento.

Modelo entidad–relación

Las relaciones representadas son FK declaradas. No se dibuja una FK inexistente entre comprobante y las cuatro tablas de origen. Los nombres de tipos se simplifican solo para Mermaid; precisión y tamaños están en el diccionario.

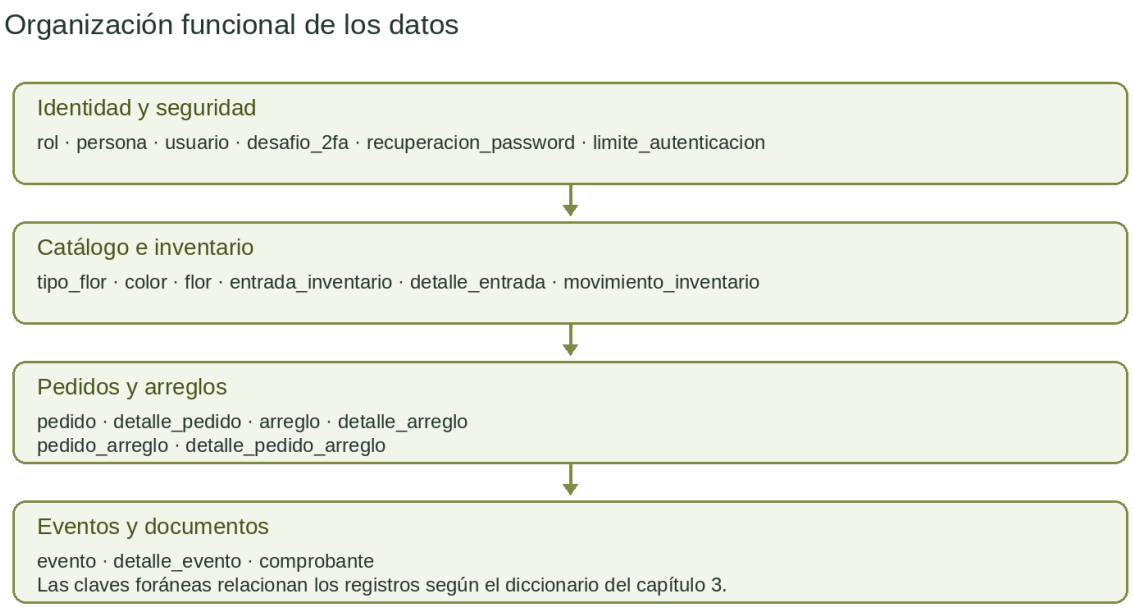


Figura T2. Agrupación funcional del modelo de datos.

El diccionario precedente detalla columnas, claves y restricciones de cada tabla.

Vistas de reportes

Vista	Fuente vigente	Significado
vw_reporte_clientes	Flor_Arte_Backend/src/main/resources/db/migration/V14__vistas_reportes_operativos.sql	Solicitudes de pedidos, arreglos y eventos; filtro de cancelación aplicado al consultar rankings.
vw_reporte_demanda_flores	Flor_Arte_Backend/src/main/resources/db/migration/V14__vistas_reportes_operativos.sql	Flores en pedidos/arreglos ENTREGADOS y eventos REALIZADOS/PAGADOS; cantidades por composición histórica.
vw_reporte_arreglos	Flor_Arte_Backend/src/main/resources/db/migration/V14__vistas_reportes_operativos.sql	Pedidos de arreglos con cantidad y total.
vw_reporte_compras	Flor_Arte_Backend/src/main/resources/db/migration/V14__vistas_reportes_operativos.sql	Cabeceras de compras por proveedor.
vw_reporte_compras_flores	Flor_Arte_Backend/src/main/resources/db/migration/V14__vistas_reportes_operativos.sql	Detalle comprado por flor/proveedor.
vw_reporte_mermas	Flor_Arte_Backend/src/main/resources/db/migration/V16__costo_historico_merma.sql	Mermas SALIDA; costo histórico o promedio de compras hasta la fecha y valoración de venta actual diferenciada.
vw_reporte_ingresos	Flor_Arte_Backend/src/main/resources/db/migration/V15__importes_eventos_y_reportes_financieros.sql	Pedidos y arreglos ENTREGADOS; eventos PAGADOS, con indicador de fecha estimada si falta fecha_pago.
vw_reporte_ingresos_diarios	Flor_Arte_Backend/src/main/resources/db/migration/V15__importes_eventos_y_reportes_financieros.sql	Agrupa ingresos por módulo/día y cuenta importes/fechas desconocidos.
vw_reporte_perdidas_diarias	Flor_Arte_Backend/src/main/resources/db/migration/V15__importes_eventos_y_reportes_financieros.sql	Unidades y costo estimado perdido por día.
vw_reporte_gastos_mensuales	Flor_Arte_Backend/src/main/resources/db/migration/V15__importes_eventos_y_reportes_financieros.sql	Compras agrupadas por mes.

Historial Flyway

Versión	Archivo	Propósito
V1	Flor_Arte_Backend/src/main/resources/db/migration/V1__crear_rol.sql	crear rol.
V2	Flor_Arte_Backend/src/main/resources/db/migration/V2__crear_persona.sql	crear persona.
V3	Flor_Arte_Backend/src/main/	crear usuario.

Versión	Archivo	Propósito
	resources/db/migration/ V3__crear_usuario.sql	
V4	Flor_Arte_Backend/src/main/ resources/db/migration/ V4__insertar_datos_iniciales.sql	insertar datos iniciales; se omiten sus valores personales y credenciales.
V5	Flor_Arte_Backend/src/main/ resources/db/migration/ V5__modulo_flores.sql	modulo flores.
V6	Flor_Arte_Backend/src/main/ resources/db/migration/ V6__modulo_inventario.sql	modulo inventario.
V7	Flor_Arte_Backend/src/main/ resources/db/migration/ V7__modulo_pedidos.sql	modulo pedidos.
V8	Flor_Arte_Backend/src/main/ resources/db/migration/ V8__agregar_fecha_pedido.sql	agregar fecha pedido.
V9	Flor_Arte_Backend/src/main/ resources/db/migration/ V9__creacion_inmutable_y_paginacion.sql	creacion inmutable y paginacion.
V10	Flor_Arte_Backend/src/main/ resources/db/migration/ V10__normalizar_contactos_opcionales.sql	normalizar contactos opcionales.
V11	Flor_Arte_Backend/src/main/ resources/db/migration/ V11__modulo_arreglos.sql	modulo arreglos.
V12	Flor_Arte_Backend/src/main/ resources/db/migration/ V12__imagen_arreglo.sql	imagen arreglo.
V13	Flor_Arte_Backend/src/main/ resources/db/migration/ V13__modulo_eventos.sql	modulo eventos.
V14	Flor_Arte_Backend/src/main/ resources/db/migration/ V14__vistas_reportes_operativos.sql	vistas reportes operativos.
V15	Flor_Arte_Backend/src/main/ resources/db/migration/ V15__importes_eventos_y_reportes_financieros.sql	importes eventos y reportes financieros.
V16	Flor_Arte_Backend/src/main/ resources/db/migration/ V16__costo_historico_merma.sql	costo historico merma.
V17	Flor_Arte_Backend/src/main/ resources/db/migration/ V17__comprobantes_operaciones.sql	comprobantes operaciones.

Versión	Archivo	Propósito
V18	Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql	seguridad correo 2fa y recuperacion.

Persistencia Java y límites del modelo

Color → color: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Color.java.

Detalle_Entrada → detalle_entrada: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Detalle_Entrada.java.

Detalle_Pedido → detalle_pedido: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Detalle_Pedido.java.

Entrada_Inventario → entrada_inventario: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Entrada_Inventario.java.

Flor → flor: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Flor.java.

Movimiento_Inventario → movimiento_inventario: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Movimiento_Inventario.java.

Pedido → pedido: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Pedido.java.

Persona → persona: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Persona.java.

Rol → rol: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/Rol.java.

TipoFlor → tipo_flor: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/TipoFlor.java.

User → usuario: Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/User.java.

Arreglos, eventos, comprobantes, autenticación por correo y reportes emplean JDBC/Map y SQL; no existe una entidad JPA para cada tabla. Fuentes: ArregloService, EventoService, ComprobanteService, DesafioAuthService, RecuperacionService, ReporteService.

Los índices adicionales de V9, V11 y V13–V18 facilitan las consultas por fecha e identificador, compras por flor, fecha de pago, comprobantes y expiración de autenticación. Algunos índices filtran filas según su estado o condición de uso. Las migraciones administran el esquema y se distinguen de los respaldos de datos.

Se identificaron 21 tablas de aplicación, 10 vistas y 25 relaciones FK; la tabla interna flyway_schema_history es administrada por Flyway, no se define como tabla de negocio.

Catálogo de índices explícitos

Además de índices implícitos de PRIMARY KEY y UNIQUE, las migraciones declaran los siguientes. Se transcriben solo DDL de índices, sin datos ni credenciales.

Fuente: V9__creacion_inmutable_y_paginacion.sql.

```
CREATE INDEX idx_pedido_fecha_id ON pedido(fecha DESC, id_pedido DESC);
CREATE INDEX idx_entrada_fecha_id ON entrada_inventario(fecha DESC, id_entrada DESC);
CREATE INDEX idx_merma_fecha_id ON movimiento_inventario(motivo, fecha DESC, id_movimiento_inventario
DESC);
```

Fuente: V11__modulo_arreglos.sql.

```
CREATE INDEX idx_pedido_arreglo_fecha ON pedido_arreglo(fecha DESC,id_pedido_arreglo DESC);
CREATE INDEX idx_pedido_arreglo_catalogo ON pedido_arreglo(id_arreglo);
```

Fuente: V13__modulo_eventos.sql.

```
CREATE INDEX idx_evento_fecha ON evento(fecha,id_evento);
CREATE INDEX idx_evento_cliente ON evento(id_cliente);
```

Fuente: V14__vistas_reportes_operativos.sql.

```
CREATE INDEX idx_detalle_entrada_flor_reporte ON detalle_entrada(id_flor,id_entrada);
```

Fuente: V15__importes_eventos_y_reportes_financieros.sql.

```
CREATE INDEX idx_evento_fecha_pago_reporte ON evento(fecha_pago) WHERE estado='PAGADO';
```

Fuente: V17__comprobantes_operaciones.sql.

```
CREATE INDEX idx_comprobante_fecha ON comprobante(emitido_en DESC,id_comprobante DESC);
```

Fuente: V18__seguridad_correo_2fa_y_recuperacion.sql.

```
CREATE INDEX idx_desafio_usuario ON desafio_2fa(id_usuario) WHERE NOT utilizado;
CREATE INDEX idx_desafio_expira ON desafio_2fa(limite_en);
CREATE INDEX idx_recuperacion_usuario ON recuperacion_password(id_usuario) WHERE NOT utilizado;
CREATE INDEX idx_recuperacion_expira ON recuperacion_password(expira_en);
CREATE INDEX idx_limite_autenticacion_inicio ON limite_autenticacion(inicio_en);
```

4. Catálogo de API REST

Alcance y convenciones

El catálogo reúne los métodos de los 17 controladores REST, incluidos los alias de ruta y las expresiones de autorización declaradas con @PreAuthorize.

La URL base se define en VITE_API_URL (frontend); no hay prefijo /api declarado en los mappings inspeccionados. SecurityConfig exige autenticación en todas las rutas salvo cinco POST públicos de autenticación. Los parámetros {id} son parámetros de ruta, no valores de negocio. En las firmas se distinguen @RequestParam, @RequestBody, @PathVariable y @Valid. Las firmas Java y DTO siguientes constituyen el contrato verificable.

El código HTTP 200 corresponde a la respuesta normal por defecto de los métodos indicados. Los cuerpos Map<String,Object> se construyen en servicios y no representan un DTO único. El manejo de errores de negocio se realiza mediante el controlador global de excepciones.

ArregloController.java

GET /arreglo/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ArregloController.java:16; método detalle.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> detalle(@PathVariable int id)
```

POST /arreglo/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ArregloController.java:17; método crear.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> crear(@Valid @RequestBody ArregloDTO dto)
```

PUT /arreglo/update/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ArregloController.java:18; método editar.

Permiso declarado: @reglasEdicion.editar('arreglos',#id).

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> editar(@PathVariable int id,@Valid @RequestBody ArregloDTO dto)
```

DELETE /arreglo/delete/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ArregloController.java:21; método eliminar.

Permiso declarado: @reglasEdicion.editar('arreglos',#id).

Respuesta normal: 200, tipo Java Map<String,String>.

Firma (parámetros y cuerpo):

```
public Map<String,String> eliminar(@PathVariable int id)
```

AuthController.java

POST /auth/login

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:32; método login.

Permiso declarado: PUBLICO (permitAll en SecurityConfig).

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> login( @Valid @RequestBody LoginUserDto dto, HttpServletRequest request)
```

POST /auth/2fa/verificar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:41; método verificar.

Permiso declarado: PUBLICO (permitAll en SecurityConfig).

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> verificar( @Valid @RequestBody Verificar dto, HttpServletRequest request)
```

POST /auth/2fa/reenviar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:48; método reenviar.

Permiso declarado: PUBLICO (permitAll en SecurityConfig).

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> reenviar( @Valid @RequestBody Reenviar dto, HttpServletRequest request)
```

POST /auth/recuperacion/solicitar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:55; método solicitar.

Permiso declarado: PUBLICO (permitAll en SecurityConfig).

Respuesta normal: 200, tipo Java Map<String, String>.

Firma (parámetros y cuerpo):

```
public Map<String, String> solicitar( @Valid @RequestBody Recuperar dto, HttpServletRequest request)
```

POST /auth/recuperacion/empleado/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:68; método solicitarEmpleado.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java Map<String, String>.

Firma (parámetros y cuerpo):

```
public Map<String, String> solicitarEmpleado(@PathVariable long id, Principal principal)
```

POST /auth/recuperacion/restablecer

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:79; método restablecer.

Permiso declarado: PUBLICO (permitAll en SecurityConfig).

Respuesta normal: 200, tipo Java Map<String, String>.

Firma (parámetros y cuerpo):

```
public Map<String, String> restablecer( @Valid @RequestBody Restablecer dto, HttpServletRequest request)
```

GET /auth/2fa/configuracion

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:88; método configuracion.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> configuracion(Principal p)
```

POST /auth/2fa/configuracion/iniciar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:94;
método configurar.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> configurar( @Valid @RequestBody Configurar dto, Principal p, HttpServletRequest
r)
```

POST /auth/2fa/configuracion/verificar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:103;
método confirmar.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> confirmar( @Valid @RequestBody Verificar dto, Principal p, HttpServletRequest r)
```

POST /auth/2fa/configuracion/reenviar

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:111;
método reenviarConfiguracion.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> reenviarConfiguracion(@Valid @RequestBody Reenviar dto, Principal p)
```

GET /auth/check-auth

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java:116;
método checkAuth.

Permiso declarado: Autenticado (SecurityConfig).

Respuesta normal: 200, tipo Java String.

Firma (parámetros y cuerpo):

```
public String checkAuth()
```

ColorController.java

GET /color/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java:26; método getAllColors.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAllColors()
```

GET /color/show/{id}, /color/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java:33; método getColorById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getColorById(@PathVariable Integer id)
```

POST /color/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java:42; método createColor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createColor(@Valid @RequestBody ColorDTO colorDto)
```

PUT /color/update/{id}, /color/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java:50; método updateColor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateColor(@PathVariable Integer id, @Valid @RequestBody ColorDTO colorDto)
```

DELETE /color/delete/{id}, /color/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ColorController.java:58; método deleteColor.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> deleteColor(@PathVariable Integer id)
```

ComprobanteController.java

POST /comprobante

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:29; método emitir.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> emitir(@Valid @RequestBody Emitir dto)
```

POST /comprobante/{id}/correo

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:36; método enviar.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, String>.

Firma (parámetros y cuerpo):

```
public Map<String, String> enviar( @PathVariable long id, @Valid @RequestBody EnviarCorreo dto,
java.security.Principal principal)
```

GET /comprobante/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:44; método detalle.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> detalle(@PathVariable long id)
```

GET /comprobante/{id}/pdf

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:49; método descargar.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java org.springframework.http.ResponseEntity<byte[]>.

Contenido application/pdf; Content-Disposition: attachment; Cache-Control: no-store.

Firma (parámetros y cuerpo):

```
public org.springframework.http.ResponseEntity<byte[]> descargar(@PathVariable long id)
```

DELETE /comprobante/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:59; método eliminar.

Permiso declarado: @reglasEdicion.editar('comprobantes', #id).

Respuesta normal: 204, tipo Java org.springframework.http.ResponseEntity<Void>.

Firma (parámetros y cuerpo):

```
public org.springframework.http.ResponseEntity<Void> eliminar(@PathVariable long id)
```

GET /comprobante/pagina

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ComprobanteController.java:66; método pagina.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> pagina( @RequestParam(defaultValue = "0") int pagina, @RequestParam(defaultValue
= "10") int tamano, @RequestParam(defaultValue = "") String tipo, @RequestParam(defaultValue = "") String
mes, @RequestParam(defaultValue = "") String q)
```

Deta_EntraController.java

GET /detalle_entrada/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:26; método getAll.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAll()
```

GET /detalle_entrada/show/{id}, /detalle_entrada/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:33 ; método getById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getById(@PathVariable Integer id)
```

GET /detalle_entrada/entrada/{idEntrada}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:42 ; método getByEntrada.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getByEntrada(@PathVariable Integer idEntrada)
```

POST /detalle_entrada/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:49 ; método create.

Permiso declarado: @reglasEdicion.editar('inventario', #dto.idEntrada).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> create(@Valid @RequestBody Detalle_EntradaDTO dto)
```

PUT /detalle_entrada/update/{id}, /detalle_entrada/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:57 ; método update.

Permiso declarado: @reglasEdicion.detalle('inventario', #id) and @reglasEdicion.editar('inventario', #dto.idEntrada).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> update(@PathVariable Integer id, @Valid @RequestBody Detalle_EntradaDTO dto)
```

DELETE /detalle_entrada/delete/{id}, /detalle_entrada/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_EntraController.java:65 ; método delete.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> delete(@PathVariable Integer id)
```

Deta_PediController.java

GET /detalle_pedido/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:26; método getAll.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAll()
```

GET /detalle_pedido/show/{id}, /detalle_pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:33; método getById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getById(@PathVariable Integer id)
```

GET /detalle_pedido/pedido/{idPedido}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:42; método getByPedido.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getByPedido(@PathVariable Integer idPedido)
```

POST /detalle_pedido/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:49; método create.

Permiso declarado: @reglasEdicion.editar('pedidos', #dto.idPedido).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> create(@Valid @RequestBody Deta_PediDTO dto)
```

PUT /detalle_pedido/update/{id}, /detalle_pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:57; método update.

Permiso declarado: @reglasEdicion.detalle('pedidos', #id) and @reglasEdicion.editar('pedidos', #dto.idPedido).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> update(@PathVariable Integer id, @Valid @RequestBody Deta_PediDTO dto)
```

DELETE /detalle_pedido/delete/{id}, /detalle_pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Deta_PediController.java:65; método delete.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> delete(@PathVariable Integer id)
```

Entra_InvController.java

GET /entrada_inventario/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:27; método getAll.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAll()
```

GET /entrada_inventario/show/{id}, /entrada_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:34; método getById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getById(@PathVariable Integer id)
```

POST /entrada_inventario/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:43; método create.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> create(@Valid @RequestBody Entrada_InventarioDTO dto)
```

POST /entrada_inventario/create-completa

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:52; método createCompleta.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 201, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createCompleta(@Valid @RequestBody Entrada_InventarioDTO dto)
```

PUT /entrada_inventario/update/{id}, /entrada_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:65; método update.

Permiso declarado: @reglasEdicion.editar('inventario', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> update(@PathVariable Integer id, @RequestBody Entrada_InventarioDTO dto)
```

DELETE /entrada_inventario/delete/{id}, /entrada_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Entra_InvController.java:72; método delete.

Permiso declarado: @reglasEdicion.editar('inventario', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> delete(@PathVariable Integer id)
```

EventoController.java

GET /evento/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:15; método detalle.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> detalle(@PathVariable int id)
```

POST /evento/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:16; método crear.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> crear(@Valid @RequestBody EventoDTO dto)
```

PUT /evento/update/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:17; método editar.

Permiso declarado: @reglasEdicion.editar('eventos',#id).

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> editar(@PathVariable int id,@Valid @RequestBody EventoDTO dto)
```

DELETE /evento/delete/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:19; método eliminar.

Permiso declarado: @reglasEdicion.editar('eventos',#id).

Respuesta normal: 200, tipo Java Map<String,String>.

Firma (parámetros y cuerpo):

```
public Map<String,String> eliminar(@PathVariable int id)
```

PUT /evento/{id}/estado

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:21; método estado.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> estado(@PathVariable int id,@Valid @RequestBody EstadoEventoDTO dto)
```

GET /evento/calendario

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:22; método calendario.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> calendario(@RequestParam int anio,@RequestParam int mes,
@RequestParam(defaultValue="") String estado,@RequestParam(defaultValue="") String q)
```

GET /evento/pagina

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/EventoController.java:24; método pagina.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> pagina(@RequestParam int anio,@RequestParam(defaultValue="0") int mes,
@RequestParam(defaultValue="") String dia,@RequestParam(defaultValue="") String
estado,@RequestParam(defaultValue="") String q, @RequestParam(defaultValue="0") int
pagina,@RequestParam(defaultValue="10") int tamano, @RequestParam(defaultValue="false") boolean
preparacion)
```

FlorController.java

GET /flor/all

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java:26;
método getAllFlores.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAllFlores()
```

GET /flor/show/{id}, /flor/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java:33;
método getFlorById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getFlorById(@PathVariable Integer id)
```

POST /flor/create

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java:42;
método createFlor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createFlor(@Valid @RequestBody FlorDTO florDto)
```

PUT /flor/update/{id}, /flor/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java:50;
método updateFlor.

Permiso declarado: @reglasEdicion.editar('flores', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateFlor(@PathVariable Integer id, @Valid @RequestBody FlorDTO florDto)
```

DELETE /flor/delete/{id}, /flor/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/FlorController.java:58; método deleteFlor.

Permiso declarado: @reglasEdicion.editar('flores', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> deleteFlor(@PathVariable Integer id)
```

GestionController.java

GET /gestion/{tipo}/pagina

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java:58; método pagina.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java Map<String, Object>.

Firma (parámetros y cuerpo):

```
public Map<String, Object> pagina(@PathVariable String tipo, @RequestParam(defaultValue="0") int pagina,
@RequestParam(defaultValue="10") int tamano, @RequestParam(defaultValue="") String q,
@RequestParam(defaultValue="") String mes, @RequestParam(defaultValue="") String estado,
@RequestParam(defaultValue="") String rol, @RequestParam(defaultValue="recientes") String orden)
```

GET /gestion/{tipo}/{id}/permiso

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java:119; método permiso.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> permiso(@PathVariable String tipo, @PathVariable Integer id)
```

GET /gestion/avisos

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java:122; método avisos.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java List<Map<String,Object>>.

Firma (parámetros y cuerpo):

```
public List<Map<String,Object>> avisos()
```

GET /gestion/resumen

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/GestionController.java:144; método resumen.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> resumen()
```

Mov_InvController.java

GET /movimiento_inventario/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:26; método getAll.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAll()
```

GET /movimiento_inventario/show/{id}, /movimiento_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:33; método getById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getById(@PathVariable Integer id)
```

GET /movimiento_inventario/flor/{idFlor}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:42; método getByFlor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getByFlor(@PathVariable Integer idFlor)
```

GET /movimiento_inventario/costo/{idFlor}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:48; método costo.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> costo(@PathVariable Integer idFlor)
```

POST /movimiento_inventario/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:57; método create.

Permiso declarado: hasRole('ADMINISTRADOR') or (hasRole('EMPLEADO') and #dto.motivo == 'MERMA' and #dto.tipoMovimiento == 'SALIDA').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> create(@Valid @RequestBody Movimiento_InventarioDTO dto)
```

PUT /movimiento_inventario/update/{id}, /movimiento_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:65; método update.

Permiso declarado: @reglasEdicion.merma(#id, #dto.motivo, #dto.tipoMovimiento).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> update(@PathVariable Integer id, @Valid @RequestBody Movimiento_InventarioDTO dto)
```

DELETE /movimiento_inventario/delete/{id}, /movimiento_inventario/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/Mov_InvController.java:73; método delete.

Permiso declarado: @reglasEdicion.merma(#id, 'MERMA', 'SALIDA').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> delete(@PathVariable Integer id)
```

PedidoArregloController.java

GET /pedido_arreglo/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoArregloController.java:17; método detalle.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> detalle(@PathVariable int id)
```

POST /pedido_arreglo/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoArregloController.java:18; método crear.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> crear(@Valid @RequestBody PedidoArregloDTO dto)
```

PUT /pedido_arreglo/update/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoArregloController.java:19; método editar.

Permiso declarado: @reglasEdicion.editar('pedidosArreglos',#id).

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> editar(@PathVariable int id,@Valid @RequestBody PedidoArregloDTO dto)
```

PUT /pedido_arreglo/{id}/estado

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoArregloController.java:22; método estado.

Permiso declarado: hasAnyRole('ADMINISTRADOR','EMPLEADO').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> estado(@PathVariable int id,@Valid @RequestBody EstadoPedidoDTO dto)
```

DELETE /pedido_arreglo/delete/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoArregloController.java:23; método eliminar.

Permiso declarado: @reglasEdicion.editar('pedidosArreglos',#id).

Respuesta normal: 200, tipo Java Map<String,String>.

Firma (parámetros y cuerpo):

```
public Map<String,String> eliminar(@PathVariable int id)
```

PedidoController.java

GET /pedido/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:28; método getAll.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAll()
```

GET /pedido/show/{id}, /pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:35; método getById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getById(@PathVariable Integer id)
```

POST /pedido/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:44; método create.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> create(@Valid @RequestBody PedidoDTO dto)
```

PUT /pedido/{id}/estado

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:52; método updateEstado.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateEstado(@PathVariable Integer id, @Valid @RequestBody EstadoPedidoDTO dto)
```

PUT /pedido/update/{id}, /pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:60; método update.

Permiso declarado: @reglasEdicion.editar('pedidos', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> update(@PathVariable Integer id, @Valid @RequestBody PedidoDTO dto)
```

PUT /pedido/update-completo/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:67; método updateCompleto.

Permiso declarado: @reglasEdicion.editar('pedidos', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateCompleto(@PathVariable Integer id, @Valid @RequestBody
UpdatePedidoCompletoDTO dto)
```

DELETE /pedido/delete/{id}, /pedido/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PedidoController.java:76; método delete.

Permiso declarado: @reglasEdicion.editar('pedidos', #id).

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> delete(@PathVariable Integer id)
```

PersonaController.java

GET /persona/all

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java:26;
método getAllPersonas.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAllPersonas()
```

GET /persona/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java:33;
método getPersonaById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getPersonaById(@PathVariable Long id)
```

POST /persona/create

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java:42;
método createPersona.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createPersona(@Valid @RequestBody PersonaDTO personaDto)
```

PUT /persona/update/{id}

Referencia:
Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java:50;
método updatePersona.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updatePersona(@PathVariable Long id, @Valid @RequestBody PersonaDTO personaDto)
```

DELETE /persona/delete/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java:58; método deletePersona.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> deletePersona(@PathVariable Long id)
```

ReporteController.java

GET /reporte/exportacion

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java:12; método exportacion.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> exportacion(@RequestParam String periodo,@RequestParam String fecha)
```

GET /reporte/operativos

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java:13; método operativos.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> operativos(@RequestParam String periodo,@RequestParam String fecha)
```

GET /reporte/financieros

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java:14; método financieros.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> financieros(@RequestParam String periodo,@RequestParam String fecha)
```

GET /reporte/mermas/pagina

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java:15; método mermas.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java Map<String,Object>.

Firma (parámetros y cuerpo):

```
public Map<String,Object> mermas(@RequestParam String periodo,@RequestParam String fecha,
@RequestParam(defaultValue="0") int pagina,@RequestParam(defaultValue="10") int tamano)
```

TipoFlorController.java

GET /tipoflor/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java:26; método getAllTiposFlor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAllTiposFlor()
```

GET /tipoflor/show/{id}, /tipoflor/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java:33; método getTipoFlorById.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getTipoFlorById(@PathVariable Integer id)
```

POST /tipoflor/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java:42; método createTipoFlor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createTipoFlor(@Valid @RequestBody TipoFlorDTO dto)
```

PUT /tipoflor/update/{id}, /tipoflor/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java:50; método updateTipoFlor.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateTipoFlor(@PathVariable Integer id, @Valid @RequestBody TipoFlorDTO dto)
```

DELETE /tipoflor/delete/{id}, /tipoflor/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/TipoFlorController.java:58; método deleteTipoFlor.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> deleteTipoFlor(@PathVariable Integer id)
```

UserController.java

POST /user/create

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java:31; método createUser.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> createUser(@Valid @RequestBody NewUserDto dto)
```

DELETE /user/delete/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java:39; método deleteUserById.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> deleteUserById(@PathVariable Long id)
```

GET /user/all

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java:47; método getAllUsers.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getAllUsers()
```

PUT /user/update/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java:54; método updateUserById.

Permiso declarado: hasRole('ADMINISTRADOR').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> updateUserById(@PathVariable Long id, @Valid @RequestBody UpdateUserDto dto)
```

GET /user/{id}

Referencia: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java:62; método getUserId.

Permiso declarado: hasAnyRole('ADMINISTRADOR', 'EMPLEADO').

Respuesta normal: 200, tipo Java ResponseEntity<?>.

Firma (parámetros y cuerpo):

```
public ResponseEntity<?> getUserId(@PathVariable Long id)
```

Contratos DTO y validaciones declaradas

Los DTO incluyen campos de entrada y salida. Un campo sin anotación no se convierte por ello en obligatorio; los servicios añaden validaciones de existencia, roles, stock y reglas financieras. Las validaciones anidadas solo se disparan automáticamente si la lista y el punto de entrada llevan @Valid. PedidoDTO.detalles y Entrada_InventarioDTO.detalles no declaran esa cascada; los servicios deben validar. Entra_InvController.update tampoco anota @Valid en el cuerpo. La validación de interfaz, DTO y base de datos se aplica en capas con responsabilidades diferentes.

ArregloDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/ArregloDTO.java.

```
public record ArregloDTO(
    @NotBlank @Size(max=100) String nombre,
    @Size(max=5000) String descripcion,
    @NotNull @DecimalMin("0") @Digits(integer=8,fraction=2) BigDecimal precio,
    @NotEmpty @Size(max=200) List<@NotNull @Valid Item> detalles,
```

```
    @Size(max=2048) String imagenUrl
) {
    public ArregloDTO(String nombre,String descripcion,BigDecimal precio,List<Item> detalles) {
        this(nombre,descripcion,precio,detalles,null);
    }
    public record Item(@NotNull @Positive Integer idFlor, @NotNull @Positive Integer cantidad) {}
}
```

ColorDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/ColorDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ColorDTO {

    private Integer idColor;

    @NotBlank(message = "El nombre del color es obligatorio")
    @Size(max = 50, message = "El nombre no puede superar los 50 caracteres")
    private String nombre;
}
```

Deta_PediDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/Deta_PediDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Deta_PediDTO {

    private Integer idDetallePedido;

    @NotNull(message = "El id de pedido es obligatorio")
    private Integer idPedido;

    @NotNull(message = "El id de flor es obligatorio")
    private Integer idFlor;

    private String nombreFlor;

    @NotNull(message = "La cantidad es obligatoria")
    @Min(value = 1, message = "La cantidad debe ser mayor a 0")
    private Integer cantidad;

    @DecimalMin(value = "0.0", inclusive = true, message = "El precio no puede ser negativo")
    private BigDecimal precio;

    private BigDecimal subtotal;
}
```

Detalle_EntradaDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/Detalle_EntradaDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Detalle_EntradaDTO {

    private Integer idDetalleEntrada;

    @NotNull(message = "El id de entrada es obligatorio")
    private Integer idEntrada;

    @NotNull(message = "El id de flor es obligatorio")
    private Integer idFlor;
```

```
    private String nombreFlor;

    @NotNull(message = "La cantidad es obligatoria")
    @Min(value = 1, message = "La cantidad debe ser mayor a 0")
    private Integer cantidad;

    @NotNull(message = "El precio de compra es obligatorio")
    @DecimalMin(value = "0.0", inclusive = true, message = "El precio de compra no puede ser negativo")
    private BigDecimal precioCompra;

    private BigDecimal subtotal;
}
```

Entrada_InventarioDTO

Fuente:
Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/Entrada_InventarioDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Entrada_InventarioDTO {

    private Integer idEntrada;

    @NotNull(message = "El proveedor es obligatorio")
    private Long idPersona;

    private String nombrePersona;

    private LocalDateTime fecha;

    @DecimalMin(value = "0.0", inclusive = true, message = "El total no puede ser negativo")
    private BigDecimal total;

    private List<Detalle_EntradaDTO> detalles;
}
```

ErrorResponse

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/ErrorResponse.java.

```
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private Map<String, String> errors;
}
```

EstadoEventoDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/EstadoEventoDTO.java.

```
public record EstadoEventoDTO(
    @NotBlank @Pattern(regexp="PENDIENTE|CONFIRMADO|TRABAJANDO|REALIZADO|PAGADO|CANCELADO") String estado,
    @DecimalMin("0") @Digits(integer=8,fraction=2) BigDecimal importe
) { public EstadoEventoDTO(String estado){this(estado,null);} }
```

EstadoPedidoDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/EstadoPedidoDTO.java.

```
public record EstadoPedidoDTO(
    @NotBlank(message = "Selecciona un estado")
    @Pattern(regexp = "PENDIENTE|CONFIRMADO|PREPARANDO|LISTO|ENTREGADO|CANCELADO", message = "Estado de
pedido inválido")
    String estado
) {}
```

EventoDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/EventoDTO.java.

```
public record EventoDTO(
    @NotBlank @Size(max=150) String nombre,
    @NotNull @Positive Long idCliente,
    @Positive Long idEmpleado,
    @NotNull LocalDateTime fecha,
    @NotNull @Min(0) @Max(365) Integer diasPreparacion,
    @Size(max=5000) String descripcion,
    @Size(max=250) String ubicacion,
    @NotEmpty @Size(max=200) List<@NotNull @Valid Item> detalles,
    @DecimalMin("0") @Digits(integer=8,fraction=2) BigDecimal importe
) {
    public EventoDTO(String nombre,Long idCliente,Long idEmpleado,LocalDateTime fecha,Integer
diasPreparacion,String descripcion,String ubicacion,List<Item> detalles){
        this(nombre,idCliente,idEmpleado,fecha,diasPreparacion,descripcion,ubicacion,detalles,null);
    }
    public record Item(@NotNull @Positive Integer idFlor,@NotNull @Positive Integer cantidad) {}
}
```

FlorDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/FlorDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class FlorDTO {

    private Integer idFlor;

    @NotNull(message = "El tipo de flor es obligatorio")
    private Integer idTipoFlor;

    private String nombreTipoFlor;

    @NotNull(message = "El color es obligatorio")
    private Integer idColor;

    private String nombreColor;

    @NotNull(message = "El precio es obligatorio")
    @DecimalMin(value = "0.0", inclusive = true, message = "El precio no puede ser negativo")
    private BigDecimal precio;

    @NotNull(message = "El stock es obligatorio")
    @Min(value = 0, message = "El stock no puede ser negativo")
    private Integer stock;

    private Boolean estado;
}
```

LoginUserDto

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/LoginUserDto.java.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class LoginUserDto {
```

```
    @NotBlank(message = "El correo electrónico es obligatorio")
    @Email(message = "El formato de correo no es válido")
    private String correo;

    @NotBlank(message = "La contraseña es obligatoria")
    private String contrasena;
}
```

Movimiento_InventarioDTO

Fuente:
Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/Movimiento_InventarioDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Movimiento_InventarioDTO {

    private Integer idMovimientoInventario;

    @NotNull(message = "El id de flor es obligatorio")
    private Integer idFlor;

    private String nombreFlor;

    @NotBlank(message = "El tipo de movimiento es obligatorio")
    private String tipoMovimiento;

    @NotNull(message = "La cantidad es obligatoria")
    @Min(value = 1, message = "La cantidad debe ser mayor a 0")
    private Integer cantidad;

    @NotBlank(message = "El motivo es obligatorio")
    private String motivo;

    private LocalDateTime fecha;

    @jakarta.validation.constraints.DecimalMin(value="0", message="El costo no puede ser negativo")
    @jakarta.validation.constraints.Digits(integer=10, fraction=6, message="El costo admite 10 enteros y 6
decimales")
    private java.math.BigDecimal costoUnitario;
    private java.math.BigDecimal perdida;

    public Movimiento_InventarioDTO(Integer id,Integer flor,String nombre,String tipo,Integer
cantidad,String motivo,LocalDateTime fecha) {
        this(id,flor,nombre,tipo,cantidad,motivo,fecha,null,null);
    }
}
```

NewUserDto

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/NewUserDto.java.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class NewUserDto {

    @NotBlank(message = "El nombre de usuario es obligatorio")
    @Size(min = 3, max = 100, message = "El nombre de usuario debe tener entre 3 y 100 caracteres")
    private String name;

    @NotBlank(message = "El correo electrónico es obligatorio")
    @Email(message = "El formato de correo no es válido")
    @Size(max = 150, message = "El correo no puede superar los 150 caracteres")
    private String email;

    @NotBlank(message = "La contraseña es obligatoria")
```

```
    @Size(min = 6, message = "La contraseña debe tener al menos 6 caracteres")
    private String password;

    @NotNull(message = "El id de persona es obligatorio")
    private Long idPersona;
}
```

PedidoArregloDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/PedidoArregloDTO.java.

```
public record PedidoArregloDTO(
    @NotNull @Positive Long idCliente,
    @Positive Long idEmpleado,
    @NotNull @Positive Integer idArreglo,
    @NotNull @Positive Integer cantidad
) {}
```

PedidoDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/PedidoDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class PedidoDTO {

    private Integer idPedido;

    @NotNull(message = "El cliente es obligatorio")
    private Long idCliente;

    private String nombreCliente;

    private Long idEmpleado;

    private String nombreEmpleado;

    private String estado;

    private LocalDateTime fecha;

    @DecimalMin(value = "0.0", inclusive = true, message = "El total no puede ser negativo")
    private BigDecimal total;

    private List<Deta_PediDTO> detalles;
}
```

PersonaDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/PersonaDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class PersonaDTO {

    private Long idPersona;

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar los 100 caracteres")
    private String nombre;

    @Size(max = 20, message = "El teléfono no puede superar los 20 caracteres")
    private String telefono;

    @Size(max = 20, message = "El DPI no puede superar los 20 caracteres")
    private String dpi;
```

```
    @Email(message = "El formato de correo no es válido")
    @Size(max = 150, message = "El correo no puede superar los 150 caracteres")
    private String correo;

    @NotNull(message = "El rol es obligatorio")
    private Long idRol;

    private String tipoRol;
}
```

SeguridadCorreoDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/SeguridadCorreoDTO.java.

```
public class SeguridadCorreoDTO {
  public record Verificar(
      @NotBlank @Size(max = 36) String challengeId,
      @NotBlank @Pattern(regexp = "[0-9]{6}", message = "Ingresa los seis dígitos del código")
          String codigo) {}

  public record Reenviar(@NotBlank @Size(max = 36) String challengeId) {}

  public record Recuperar(@NotBlank @Email @Size(max = 150) String correo) {}

  public record Restablecer(
      @NotBlank @Size(max = 100) String token,
      @NotBlank @Size(min = 8, max = 72) String contrasena,
      @NotBlank @Size(max = 72) String confirmacion) {}

  public record Configurar(
      @NotNull Boolean habilitado, @NotBlank @Size(max = 256) String contrasena) {}
}
```

TipoFlorDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/TipoFlorDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class TipoFlorDTO {

    private Integer idTipoFlor;

    @NotBlank(message = "El nombre del tipo de flor es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar los 100 caracteres")
    private String nombre;

    private String descripcion;

    private String imagenUrl;
}
```

UpdatePedidoCompletoDTO

Fuente:
Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/UpdatePedidoCompletoDTO.java.

```
public record UpdatePedidoCompletoDTO(
        @NotNull @Positive Long idCliente,
        @Positive Long idEmpleado,
        @NotBlank String estado,
        @NotEmpty List<@NotNull @Valid Item> detalles
) {
    public record Item(
            @Positive Integer idDetallePedido,
            @NotNull @Positive Integer idFlor,
            @NotNull @Positive Integer cantidad,
            @NotNull @DecimalMin("0.00") @Digits(integer = 8, fraction = 2) BigDecimal precio
```

```
    ) {}
}
```

UpdateUserDto

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/UpdateUserDto.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UpdateUserDto {

    private Long idUsuario;

    @NotBlank(message = "El correo electrónico es obligatorio")
    @Email(message = "El formato de correo no es válido")
    @Size(max = 150, message = "El correo no puede superar los 150 caracteres")
    private String email;

    private String password;

    private String token;

    @NotNull(message = "El estado es obligatorio")
    private Boolean estado;

    @NotNull(message = "El id de persona es obligatorio")
    private Long idPersona;

    private String nombrePersona;
    private String rolPersona;
}
```

UserListDTO

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/UserListDTO.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserListDTO {

    private Long idUsuario;
    private String name;
    private String email;
    private Boolean estado;

    private Long idPersona;
    private String nombre;
    private String telefono;
    private String dpi;
    private String rol;
}
```

UserProfileDto

Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/dtos/UserProfileDto.java.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class UserProfileDto {
    private String nombre;
    private String telefono;
    private String dpi;
    private String email;
    private Boolean estado;
    private Long idPersona;
    private String rol;
```

```
    private String user;
}
```

Cuerpos locales de comprobante

ComprobanteController.Emitir: tipo obligatorio y idOrigen >= 1. Tipos admitidos por servicio: PEDIDO, ARREGLO, EVENTO, INVENTARIO. EnviarCorreo: correo obligatorio, formato email, máximo 150 caracteres. Referencia: controllers/ComprobanteController.java (enlaces del inventario).

Respuestas de negocio y filtros

Login final: requires2fa:false, token, nombre, tipoUsuario, id_usuario, estado, message. Desafío: requires2fa:true, challengeId, expiresInSeconds, resendAfterSeconds, message, sin JWT definitivo. Fuente: services/DesafioAuthService.java, src/types/auth.ts.

Gestión paginada: contenido, pagina, tamano, totalElementos, totalPaginas, resumen, horaServidor; filas con permisos/plazo cuando procede. tipo permitido: arreglos, pedidosArreglos, tipoflor, flores, pedidos, inventario, mermas, personas, usuarios. Página >= 0, tamaño 1–50, búsqueda <= 100 caracteres; mes AAAA-MM. Usuarios solo administrador incluso bajo controlador compartido. Fuentes: GestionController.consulta/pagina, src/services/gestionService.ts.

Eventos: calendario mensual por año/mes, estado/búsqueda; agenda por mes, año o día y fecha de preparación. Página 0+, tamaño 1–50; años 1900–9999. Fuentes: EventoController, EventoService.filtro/pagina/calendario.

Reportes: periodo=DIA|SEMANA|MES|ANIO, fecha=AAAA-MM-DD; semana inicia lunes. Años 1900–9998. Exportación devuelve JSON, el frontend genera PDF/Excel. Máximo 10 000 mermas en detalle exportado. Fuentes: ReporteService, PeriodoReporte, src/services/exportarDocumentos.ts.

Comprobantes: número/identificador, tipo, origen, documento JSON serializado, emisor, fecha y permisos. El documento es una instantánea; no contiene una FK polimórfica hacia la operación. Fuente: ComprobanteService, V17.

Errores reales

Fuente: exceptions/GlobalExceptionHandler.java, exceptions/FlujoAuthException.java, jwt/JwtEntryPoint.java, jwt/JwtAuthenticationFilter.java.

HTTP	Situación y contrato
400	DTO inválido: timestamp,status,error,message,errors con mensajes por campo; argumento/regla inválida: message; enlace inválido: error=ENLACE_INVALIDO y message.
401	Credenciales incorrectas, OTP inválido/vencido, JWT ausente/inválido; autenticación por EntryPoint puede tener otro formato que el Advice.
403	Método/rol no autorizado, plazo agotado; cuenta inactiva durante filtro puede devolver error=CUENTA_INACTIVA.

HTTP	Situación y contrato
429	Frecuencia de solicitudes, espera o máximo de reenvíos; error y message.
503	Servicio de correo no configurado, autenticación SMTP fallida o indisponibilidad; error y message. Recuperación asíncrona mantiene respuesta genérica, no garantiza entrega.
500	Error genérico; en /auth/ se oculta el detalle de errores RuntimeException no contemplados.

No hay paginación automática en los endpoints /all: devuelven listas completas. Las pantallas nuevas utilizan /gestion/.../pagina; algunos selectores todavía usan servicios de catálogo.

Ejemplos exclusivamente ficticios

No contienen credenciales, códigos, JWT ni enlaces de recuperación. En peticiones protegidas se requiere una cabecera Authorization obtenida por el flujo real; se omite aquí su valor.

```
GET /gestion/flores/pagina?pagina=0&tamano=10&q=rosa
GET /reporte/financieros?periodo=MES&fecha=2026-10-01
GET /evento/calendario?anio=2026&mes=10
```

```
{"nombre":"Contacto de ejemplo","telefono":null,"dpi":null,"correo":null,"idRol":3}
```

```
{"idCliente":100,"idEmpleado":null,"detalles":[{"idFlor":200,"cantidad":2,"precio":15.00}]}
```

El segundo cuerpo corresponde a POST /pedido/create; los identificadores deben existir en una base de ensayo. El servidor calcula subtotales y totales según su servicio. Un idPedido del detalle no es necesario cuando se crea junto con la cabecera; no confundir con POST /detalle_pedido/create.

```
{"tipo":"PEDIDO","idOrigen":300}
```

Corresponde a POST /comprobante. La operación debe existir, tener importe y detalle y no estar cancelada.

Observaciones de mantenimiento

La administración paginada de cuentas exige ADMINISTRADOR. Las rutas /user/all y /user/{id} también admiten EMPLEADO. Esta diferencia forma parte del contrato de autorización y debe considerarse al integrar clientes que llamen directamente a la API.

Total: 98 métodos de controlador, 122 combinaciones método/ruta contando aliases.

Guía de cuerpos de respuesta por familia

La tabla siguiente describe las formas JSON construidas por los controladores y servicios. Los campos calculados y valores nulos corresponden a los DTO y consultas SQL de cada módulo.

Familia / operación	Cuerpo exitoso y evidencia
Flor: all / show / create / update / delete	Lista FlorDTO / FlorDTO / {message,flor} / {message,flor} / {message}; FlorController y FlorService.
Color	Lista ColorDTO / ColorDTO / {message,color} en create/update / {message} en delete; ColorController.
TipoFlor	Lista TipoFlorDTO / TipoFlorDTO / {message,tipoFlor} en create/update / {message} en delete; TipoFlorController.

Familia / operación	Cuerpo exitoso y evidencia
Persona	Lista PersonaDTO / PersonaDTO / {message,persona} en create/update / {message} al eliminar; PersonaController.
Usuario	Lista UserListDTO en all; entidad User en consulta individual; {message,user} en create/update; {message} al eliminar. Entidad User oculta password/token/authVersion al serializar. UserController, UserListDTO y User.
Pedido	Lista PedidoDTO / PedidoDTO con detalles / {message,pedido} en create/update/update-completo/estado / {message} al eliminar; PedidoController.
Detalle de pedido	Lista Deta_PediDTO en all y por pedido; DTO individual; {message,detalle} en create/update; {message} al eliminar; Deta_PediController.
Entrada de inventario	Lista Entrada_InventarioDTO / DTO con detalles; {message,entrada} en create y create-completa (este último HTTP 201); update devuelve DTO directamente; delete {message}. Entra_InvController.
Detalle de entrada	Lista Detalle_EntradaDTO en all y por entrada; DTO individual; {message,detalle} al crear/actualizar; {message} al eliminar. Deta_EntraController.
Movimiento	Lista Movimiento_InventarioDTO en all/por flor; DTO individual; {message,movimiento} al crear/actualizar; {message} al eliminar; costo {costoUnitario} (puede ser null). Mov_InvController.
Arreglo	Mapa de arreglo con identificador, nombre, descripción, imagenUrl, precio, composición y capacidad; create/update devuelve detalle; delete {message}. ArregloService.arreglo/guardarArreglo.
Pedido de arreglo	Mapa con idPedidoArreglo, idCliente/nombreCliente, idEmpleado/nombreEmpleado, idArreglo/nombreArreglo, cantidad, precioUnitario, total, fecha, estado, imagenUrl y detalles; create/update/estado devuelve pedido; delete {message}. ArregloService.pedido.
Evento	Mapa con datos del evento, cliente/responsable, preparación, importe/pago, estado, reserva, flores, estadosPermitidos y permisos; create/update/estado retorna detalle. Calendario {dias,preparacion,anio,mes}; página con contenido, paginación, resumen y horaServidor. EventoService.
Comprobante	{id,tipo,idOrigen,documento,emitidoEn,emitidoPor,puedeEliminar,editableHasta,horaServidor}; documento se devuelve como JSON serializado que React parsea. PDF bytes; correo {message}; delete sin cuerpo HTTP 204. ComprobanteService/Controller.
Gestión	Página con contenido/resumen/tiempo; permiso

Familia / operación	Cuerpo exitoso y evidencia
	{puedeEditar,editableHasta,horaServidor}; avisos lista con id/tipo/titulo/detalle/ruta; resumen mapa de contadores según rol. GestionController/ReglasEdicion.
Reporte financiero	{periodo,ingresos,totalIngresos,gastos,perdidas,balance, floresDesechadas}. Balance = totalIngresos − gastos. ReporteService.financieros.
Reporte operativo	periodo, floresDemandadas, clientesPEDIDOS/clientesARREGLOS/clientesEVENTOS, arreglosSolicitados, floresCompradas, proveedores, gastosMensuales, gastos. ReporteService.operativos.
Reporte exportación	{finanzas,operativos,mermas}; no devuelve un archivo PDF/Excel. ReporteService.exportacion y generador frontend.
Autenticación	Sesión/desafío descritos antes; configuracion {habilitado}; solicitar/restablecer recuperación {message}; check-auth texto Autenticado. AuthController/DesafioAuthService.

Las fuentes completas de cada familia están enlazadas en el inventario y en el catálogo DTO. El Advice es compartido, pero filtro/EntryPoint y errores de parseo pueden tener otro formato. El manejo genérico de excepciones SQL puede devolver códigos HTTP 400 o 500; los errores específicos se tratan según las reglas del servicio.

5. Instalación y despliegue

Entorno de instalación

El repositorio incluye Dockerfiles para frontend y backend, configuración de Nginx y compose.yaml. El entorno de producción utiliza Caddy para la entrada HTTPS; la organización de sus componentes se describe en el capítulo 10.

Requisitos de ejecución

El backend utiliza JDK 21 y Maven Wrapper. La base PostgreSQL debe admitir JSONB, PL/pgSQL y los objetos definidos por Flyway. El frontend utiliza Node.js, npm, React, TypeScript y Vite; la instalación de dependencias se realiza con el lockfile del proyecto.

Fuentes: Flor_Arte_Backend/pom.xml, Flor_Arte_Backend/.mvn/wrapper/maven-wrapper.properties, Flor_Arte_Fronted/package.json, Flor_Arte_Fronted/package-lock.json, Flor_Arte_Backend/src/main/resources/db/migration/V17__comprobantes_operaciones.sql, Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql.

Variables y propiedades (sin valores privados)

Nombre	Proceso	Uso
VITE_API_URL	Compilación/Vite frontend	URL base pública obligatoria, sin sufijos auth/login; Flor_Arte_Fronted/src/config/api.ts, plantilla de configuración. Se incorpora al build y requiere recompilar si cambia.
SPRING_DATASOURCE_URL	Backend	Override estándar de spring.datasource.url; propiedad presente en configuración privada local. URL/host reales omitidos.
SPRING_DATASOURCE_USERNAME / SPRING_DATASOURCE_PASSWORD	Backend	Overrides de credenciales de datasource. No colocar en VITE_.
JWT_SECRET	Backend	Override de jwt.secret; requerido por Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtUtil.java. Debe ser privado y apto para firma HMAC.
JWT_EXPIRATION	Backend	jwt.expiration en segundos, valor local inspeccionado 3600.
MAIL_USERNAME / MAIL_PASSWORD	Backend	Usuario y contraseña de aplicación SMTP; Flor_Arte_Backend/src/main/resources/auth-mail.properties, Flor_Arte_Backend/src/main/java/com/florarte/backend/config/CorreoCo

Nombre	Proceso	Uso
		nfig.java.
MAIL_FROM	Backend	Remitente autorizado; por defecto usa MAIL_USERNAME.
AUTH_RECOVERY_URL	Backend	URL completa de /restablecer-contrasena, HTTPS salvo localhost HTTP; sin query/fragmento. Default local 5173.
AUTH_CODE_SECRET	Backend	HMAC OTP/límites, mínimo 32 bytes. Por defecto referencia clave JWT; se recomienda separar.
CORS_ALLOWED_ORIGINS	Backend	Lista de orígenes separados por comas; default localhost:5173.
SERVER_PORT	Backend	Override opcional estándar para puerto HTTP; no está fijado en las fuentes privadas inspeccionadas.

Las propiedades del backend se suministran al proceso de Spring Boot mediante su configuración externa. Vite incorpora las variables públicas de compilación del frontend. Los parámetros privados de conexión y firma se administran fuera del control de versiones.

Procedimiento de desarrollo

Obtener el repositorio Flor_Arte y seleccionar la rama de trabajo correspondiente.

Preparar una instancia PostgreSQL y una base vacía controlada. DataBase/script.sql solo contiene creación de la base florarte; no contiene el esquema funcional. No ejecutar sobre una instancia compartida sin revisar su contexto.

Configurar el proceso backend con datasource, clave JWT, correo y orígenes. El application.properties local está ignorado en Git: un clon limpio necesita suministrar las propiedades. Configuración mínima no secreta de migraciones: spring.jpa.hibernate.ddl-auto=validate, spring.flyway.enabled=true, spring.flyway.locations=classpath:db/migration; importar auth-mail.properties (CorreoConfig también declara PropertySource). Mantener validate; no cambiar a create/create-drop sobre datos existentes.

Revisar V4: inserta roles y cuentas/personas iniciales. No se reproducen sus datos. Acordar un procedimiento seguro de bootstrap y revisar las cuentas antes de exponer una base nueva. No alterar migraciones aplicadas para cambiar esos datos.

Iniciar backend desde su carpeta:

```
cd Flor_Arte_Backend
./mvnw spring-boot:run
```

Windows: mvnw.cmd spring-boot:run. Maven y dependencias requieren conexión durante la primera descarga. Flyway aplica migraciones antes de validar entidades; no constituye un respaldo.

Preparar frontend:

```
cd Flor_Arte_Fronted
npm ci
npm run dev
```

Configurar VITE_API_URL con la dirección de la API y reiniciar Vite después de cambiar las variables. El servidor de desarrollo utiliza habitualmente el puerto 5173 y Spring Boot el 8080; CORS debe admitir el origen utilizado.

Abrir / (landing), después «Iniciar sesión» → /login. Validar acceso con cuenta de prueba, datos de negocio y restricciones antes de usar información real.

Build y ejecución de artefactos

```
# Desde Flor_Arte_Fronted
npm run build
# Desde Flor_Arte_Backend, en otra terminal
./mvnw package
java -jar target/flor_arte_backend-0.0.1-SNAPSHOT.jar
```

El JAR se genera según artifactId y version del proyecto. npm run preview permite revisar el build del frontend; para el servicio web se utiliza Nginx y la configuración de proxy descrita en el capítulo 10.

El servidor de archivos debe devolver index.html para las rutas internas de React y conservar separados los recursos estáticos y la API. El backend requiere un proceso supervisado y configuración externa. CORS y la URL de recuperación deben corresponder al dominio de la instalación.

Gmail y puertos/redes

El servicio de correo utiliza Gmail SMTP, puerto 587 y STARTTLS obligatorio. Incluye verificación de identidad del servidor y límites de 5000 ms para conexión, lectura y escritura. La cuenta remitente y su credencial de aplicación se suministran como parámetros privados.

La solicitud de recuperación se procesa de forma asíncrona. La aceptación por SMTP corresponde al envío al servidor de correo; la recepción final depende del servicio del destinatario.

Respaldo, actualización y reversión

La actualización de una instalación debe comenzar con un respaldo consistente y una comprobación de restauración. Las migraciones nuevas se aplican en orden, sin modificar versiones ya utilizadas. La reversión de una imagen o un JAR debe considerar la compatibilidad del esquema de datos.

No hay migraciones reversas disponibles; volver a un JAR anterior puede ser incompatible con el esquema. La actualización conserva los volúmenes y el esquema public de la base existente. Las pruebas que reinician esquemas solo pueden usarse en base efímera dedicada.

6. Seguridad, pruebas y CI/CD

Identidad y autorización implementadas

Spring Security usa STATELESS, filtro JWT antes del filtro de usuario/contraseña y method security. Solo cinco POST son públicos: login, verificar/reenvío 2FA de login, solicitar recuperación y restablecer. GET /auth/check-auth exige autenticación. CSRF está deshabilitado; CORS limita orígenes configurados y permite GET/POST/PUT/DELETE/OPTIONS, cabeceras Authorization y Content-Type. La autenticación utiliza el token Bearer enviado en las solicitudes. Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/config/SecurityConfig.java.

Contraseñas se verifican mediante AuthenticationManager/UserDetailsService y BCryptPasswordEncoder. La autoridad se obtiene del rol actual de persona. Rol único por cuenta: ADMINISTRADOR, EMPLEADO, CLIENTE o PROVEEDOR en el enum, pero el alta normal solo admite personas de los dos primeros. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/config/ApplicationConfig.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/AuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/UserService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/enums/RoleList.java.

Política	Implementación real y límites
Administración de cuentas	Crear/actualizar/eliminar solo ADMINISTRADOR; consultar /user/all y /user/{id} también permite EMPLEADO. Gestión paginada de usuarios solo administrador.
Contactos	Ambos leen; empleado solo crea/edita CLIENTE/PROVEEDOR y no puede modificar personas internas. El endpoint de eliminación es exclusivo de administrador. PersonaService.findAll/findById no restringe lectura como sí lo hace el listado paginado de gestión.
Operación compartida	Ambos gestionan flores, inventario, pedidos, arreglos, eventos y recibos con restricciones del servicio.
Plazo	ReglasEdicion: empleado solo antes de creado_en +1800 s, nunca en futuro; administrador sin ese plazo. No verifica propiedad del registro. Catálogos y personas tienen permisos propios.
Estado	Pedido, pedido de arreglo y evento tienen endpoints de estado para ambos roles sin límite de edición de 30 minutos; las transiciones de evento sí tienen reglas.
Reportes	Solo ADMINISTRADOR en controlador y rutas de React.
Datos sensibles al serializar	User.password y token WRITE_ONLY, authVersion JsonIgnore; no significa que otros datos de persona estén ocultos.

Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/UserController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/PersonaController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PersonaService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReglasEdicion.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/ReporteController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/entities/User.java; contratos completos en 04.

JWT, revocación y almacenamiento cliente

JWT firmado con clave HMAC privada, subject correo y claims role/authVersion; filtro compara versión persistida y usuario activo y recarga las autoridades actuales. V18 revoca ante cambios de contraseña/email/estado/persona/configuración 2FA; tokens sin authVersion se consideran versión 0 por compatibilidad. No existe una tabla de blacklist de JWT ni revocación al cerrar sesión; logout elimina copia del navegador. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtUtil.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/jwt/JwtAuthenticationFilter.java, Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql.

React almacena JWT final y datos mínimos de sesión en localStorage. Esto permite persistencia entre recargas pero expone esa copia ante XSS; el token no se almacena en una cookie HttpOnly. El rol local solo controla navegación. Fuente: Flor_Arte_Fronted/src/services/authService.ts, Flor_Arte_Fronted/src/components/PrivateRoute.tsx, Flor_Arte_Fronted/src/services/apiService.ts.

Segundo factor por correo

La verificación en dos pasos se configura por cuenta desde Mi perfil. SecureRandom genera códigos de seis dígitos; un UUID identifica el desafío. El servicio conserva el HMAC-SHA256 del código y compara los resultados mediante MessageDigest.isEqual.

El código vive 300 segundos; desafío tiene máximo absoluto 900 segundos; máximo cinco fallos acumulados y tres reenvíos con espera de 60 segundos. Reenviar sustituye hash/código sin reiniciar intentos; nuevo desafío invalida anteriores del usuario. La verificación bloquea filas y comprueba propósito, usuario, authVersion, expiración y utilizado; solo al completar entrega JWT. Si falla SMTP síncrono, invalida desafío y devuelve error de correo. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/services/DesafioAuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/SecretosAuth.java, V18, Flor_Arte_Fronted/src/components/login/CodigoCorreo.tsx, Flor_Arte_Fronted/src/components/login/SeguridadCorreo.tsx.

Recuperación y protección del enlace

La recuperación genera un token aleatorio de 32 bytes en Base64URL y conserva su hash SHA-256. El enlace dura 900 segundos y tiene un solo uso; una nueva solicitud invalida los enlaces anteriores. El servicio verifica estado y versión de cuenta, actualiza la contraseña con

BCrypt y revoca los accesos anteriores. El restablecimiento exige al menos ocho caracteres y un máximo de 72 bytes UTF-8; el alta normal de cuentas exige un mínimo de seis caracteres.

Solicitud pública responde genéricamente, incluso para email desconocido/inactivo. Administrador puede solicitar para empleado activo por ID; usa el correo de acceso guardado, no un destinatario enviado por frontend. No cambia contraseña hasta que el empleado use el enlace. Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/RecuperacionService.java, Flor_Arte_Fronted/src/pages/admin/user/UpdateUser.tsx.

Enlace con fragmento #token=...; index.html lo retira antes de cargar la aplicación y lo entrega transitoriamente a la pantalla. Política no-referrer, fuentes/imagenes locales para autenticación y token en memoria. URL backend exige HTTPS salvo localhost y no acepta userinfo, query ni fragmento. Fuentes: Flor_Arte_Fronted/index.html, Flor_Arte_Fronted/src/pages/auth/RestablecerContrasena.tsx, Flor_Arte_Fronted/src/styles/fuentes.css. Las imágenes externas de la landing no se deben incorporar a recuperación.

Límites de frecuencia persistidos

Todos los periodos de la tabla son ventanas de 900 segundos. Se contabilizan solicitudes, incluso fallidas. La clave del contador se guarda mediante HMAC, no IP/email en claro. getRemoteAddr() identifica origen; no hay adaptación verificada de forwarded headers/proxy. Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/controllers/AuthController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/LimiteAuthService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteCorreoService.java.

Acción/clave	Máximo
Login por IP / por correo	30 / 10
Verificar OTP de login por IP	60
Reenvío login por IP	30
Recuperación por IP / por correo	10 / 3
Solicitud administrativa por administrador / empleado destino	10 / 3
Restablecer por IP	30
Iniciar configuración 2FA por usuario	5
Verificar configuración por usuario	30
Enviar comprobante por usuario / por comprobante	30 / 3

El reenvío de configuración usa la espera y máximo por desafío, sin un límite adicional consumido en su controlador. Limpieza programada cada hora elimina registros de seguridad expirados con antigüedad adicional de un día. No son logs de auditoría permanentes. Fuente: Flor_Arte_Backend/src/main/java/com/florarte/backend/services/LimiteAuthService.java.

Correo, errores y secretos

JavaMailSender, Gmail SMTP 587 y STARTTLS obligatorio con verificación de identidad y timeouts. Plantillas HTML escapan datos variables; logo inline CID y PDF como adjunto. Recuperación usa @Async con executor de dos hilos y cola de 100 elementos; no es una cola persistente. Los errores se registran por categoría sin dirección/código/token en el servicio. Fuentes: Flor_Arte_Backend/src/main/java/com/florarte/backend/services/CorreoService.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/config/CorreoConfig.java.

Las credenciales de base de datos, correo y firma se administran como configuración privada. Los archivos de configuración local se excluyen del control de versiones; las variables públicas de Vite contienen únicamente datos necesarios para el cliente.

Pruebas del sistema

Las pruebas del backend cubren reglas de negocio, autenticación, recuperación, comprobantes y operaciones transaccionales. Las pruebas de navegador cubren formularios y navegación. Las tablas siguientes relacionan los archivos y sus áreas de cobertura.

Fuente backend	Cobertura declarada por pruebas
Flor_Arte_Backend/src/test/java/com/florarte/backend/SeguridadCorreoIntegrationTest.java	Login, OTP único/vencido/intentos/reenvíos, recuperación, revocación, límites, permisos, recibos/correo con mocks; requiere base temporal explícita.
Flor_Arte_Backend/src/test/java/com/florarte/backend/FlorArteBackendApplicationTests.java	Arranque y transacciones reales de arreglos/eventos; condicionado a base temporal.
Flor_Arte_Backend/src/test/java/com/florarte/backend/controllers/GestionTest.java	Consultas SQL, filtros/páginas/resúmenes; puede reiniciar schema public en la base de ensayo.
Flor_Arte_Backend/src/test/java/com/florarte/backend/controllers/PedidoEstadoPermisosTest.java	Estado de pedidos y restricciones de roles sin plazo.
Flor_Arte_Backend/src/test/java/com/florarte/backend/services/ReglasEdicionTest.java	Plazo, límites temporales y reglas de edición.
Flor_Arte_Backend/src/test/java/com/florarte/backend/services/PersonaOpcionalesTest.java	Contactos opcionales y normalización.
Flor_Arte_Backend/src/test/java/com/florarte/backend/services/MermaStockTest.java, Flor_Arte_Backend/src/test/java/com/florarte/backend/services/PedidoCompletoTest.java	Stock y actualización de detalles.
Flor_Arte_Backend/src/test/java/com/florarte/backend/services/ArregloTest.java, Flor_Arte_Backend/src/test/java/com/florarte/backend/services/EventoTest.java, Flor_Arte_Backend/src/test/java/com/florarte/backend/services/ReporteTest.java	Recetas/pedidos, reserva/transiciones, agregaciones y periodos; revisar cada clase para sus precondiciones de DB.
Flor_Arte_Backend/src/test/java/com/florarte/backend/services/ComprobantePdfTest.java, Flor_Arte_Backend/src/test/java/com/florarte/backend/services/CorreoServiceTest.java	PDF, layout/paginación/adjuntos y envío simulado.

Frontend (Flor_Arte_Fronted/package.json y carpeta tests):

Script	Fuentes / cobertura
npm run test:auth	Flor_Arte_Fronted/tests/auth.browser.mjs: flujo login/2FA/recuperación/perfil con API simulada.
npm run test:recibos	Flor_Arte_Fronted/tests/recibos-correo.browser.mjs: formulario, correo, errores y móvil, sin envíos reales.
npm run test:recibos:acciones	Flor_Arte_Fronted/tests/recibos-acciones.browser.mjs: permisos, confirmación y PDF con API simulada.
npm run test:formularios	Flor_Arte_Fronted/tests/regreso-formularios.mjs y Flor_Arte_Fronted/tests/formularios.browser.mjs: rutas de regreso y flujos simulados.
npm run test:login:estados	Flor_Arte_Fronted/tests/login-estados.browser.mjs: mensajes de error, recuperación y confirmación de estado.
npm run build / npm run lint	Compilación y revisión estática; no equivalen a pruebas funcionales.

Condiciones de integración backend: varias clases requieren -Dflorarte.test.db apuntando a jdbc:postgresql://localhost:PUERTO/florarte_test o 127.0.0.1, usuario de ensayo, con esquema desechable. No proporcionar base de negocio ni cambiar su nombre para pasar una protección. La omisión de esas pruebas no es un aprobado. Fuentes: anotaciones EnabledIfSystemProperty y métodos de preparación en las clases indicadas.

Integración continua

GitHub Actions dispone de workflows para frontend, backend y construcción de imágenes. Las tareas de frontend ejecutan ESLint, TypeScript y Vite; las de backend emplean Maven y una base PostgreSQL temporal para integración. El capítulo 9 desarrolla este flujo.

7. Matriz de trazabilidad

Organización de la trazabilidad

Los identificadores RF organizan las capacidades funcionales de FlorArte. La matriz relaciona cada función con sus componentes y rutas, para facilitar la consulta entre el manual de usuario y el catálogo técnico.

Los métodos HTTP, los contratos DTO y los permisos se desarrollan en el capítulo 4. Los procedimientos de operación se encuentran en el Manual de Usuario.

Matriz

Función	Componentes	Ruta / endpoint
RF01 Presentación pública	Flor_Arte_Fronted/src/pages/ Landing.tsx, AppRoutes	/ → /login
RF02 Ingreso por cuenta activa	Flor_Arte_Backend/src/main/java/ com/florarte/backend/controllers/ AuthController.java, Flor_Arte_Backend/src/main/java/com/florarte/backend/services/AuthService.java, UserService	POST /auth/login, /login
RF03 Sesión JWT y rol	Flor_Arte_Backend/src/main/java/ com/florarte/backend/jwt/ JwtAuthenticationFilter.java, SecurityConfig, Flor_Arte_Fronted/src/components/PrivateRoute.tsx	Rutas protegidas, /auth/check-auth
RF04 Segundo factor opcional	Flor_Arte_Backend/src/main/java/ com/florarte/backend/services/ DesafioAuthService.java, Flor_Arte_Backend/src/main/resources/db/migration/V18__seguridad_correo_2fa_y_recuperacion.sql	POST /auth/2fa/verificar, reenvío, perfil
RF05 Recuperación personal	Flor_Arte_Backend/src/main/java/ com/florarte/backend/services/ RecuperacionService.java	/recuperar-contrasena, /restablecer-contrasena
RF06 Solicitud del administrador para empleado	AuthController, RecuperacionService, Flor_Arte_Fronted/src/pages/admin/ user/UpdateUser.tsx	POST /auth/recuperacion/empleado/{id}
RF07 Personas con datos opcionales	Flor_Arte_Backend/src/main/java/ com/florarte/backend/services/ PersonaService.java, PersonaDTO, Flor_Arte_Fronted/src/utils/persona.ts	/persona/create, /persona/update/{id}
RF08 Gestión de cuentas	Flor_Arte_Backend/src/main/java/ com/florarte/backend/services/ UserService.java, UserController	/admin/usuarios, /user/*

Función	Componentes	Ruta / endpoint
RF09 Flores	FlorController/FlorService y páginas shared/flores	/flor/*, BASE/flores
RF10 Tipos/colores	TipoFlorController/ColorController	BASE/flores/tipoflor, BASE/flores/color
RF11 Compras de inventario	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Entra_InvService.java, Deta_EntraService	POST /entrada_inventario/create-completa, inventario
RF12 Merma con costo	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/Mov_InvService.java, V16	/movimiento_inventario/*, formulario Merma
RF13 Plazo de edición	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReglasEdicion.java, Flor_Arte_Fronted/src/components/shared/PermisoEdicion.tsx	/gestion/{tipo}/{id}/permiso, PUT/DELETE
RF14 Pedidos y cantidades	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/PedidoService.java, DTO Integer	/pedido/create, /pedido/update-completo/{id}
RF15 Cambio de estado sin plazo	PedidoController/PedidoArregloController	PUT /{id}/estado
RF16 Receta e imagen de arreglo	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ArregloService.java, V11/V12	/arreglo/*, catálogo y detalle
RF17 Venta de arreglos	ArregloService/pedido histórico	/pedido_arreglo/*
RF18 Calendario de eventos	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/EventoService.java, Flor_Arte_Fronted/src/pages/shared/eventos/Index.tsx	/evento/calendario, /evento/pagina
RF19 Reserva y estados de evento	EventoService, V13/V15	/evento/create, /{id}/estado
RF20 Reportes SQL y filtros	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ReporteService.java, V14–V16	/reporte/financieros, /operativos, /mermas/pagina
RF21 Reportes solo administrador	ReporteController, AdminRoutes	/admin/reportes
RF22 Snapshot de comprobante	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteService.java, V17	POST /comprobante, /comprobante/pagina
RF23 PDF de comprobante	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobantePdfService.java	GET /comprobante/{id}/pdf
RF24 Envío PDF por correo	Flor_Arte_Backend/src/main/java/com/florarte/backend/services/ComprobanteCorreoService.java, CorreoService	POST /comprobante/{id}/correo
RF25 Eliminar comprobante	ComprobanteService, ReglasEdicion	DELETE /comprobante/{id}

Función	Componentes	Ruta / endpoint
RF26 Exportar reportes	Flor_Arte_Fronted/src/services/ exportarDocumentos.ts	/reporte/exportacion, botones PDF/Excel
RF27 Inicio/avisos	GestionController, headers, ActividadInicio	/gestion/resumen, /gestion/avisos
RF28 Regreso tras guardar	Flor_Arte_Fronted/src/utils/ regresoFormulario.ts, useVolverFormulario	Create/update y pestañas
RF29 Perfil sin foto	Flor_Arte_Fronted/src/pages/ shared/perfil/Perfil.tsx, EditPerfil	/admin/perfil, /empleado/perfil
RF30 Responsive y modales	CSS, PanelLayout, Modal	Módulos de panel y landing
RF31 CI/CD, Docker, AWS	No hay archivos que acrediten instalación	No endpoint/pantalla atribuible

BASE representa /admin o /empleado; depende de la ruta del rol, no es una ruta literal.

8. Repositorio y código fuente

Organización del monorepositorio

El repositorio Flor_Arte reúne el frontend React y el backend Spring Boot, junto con las migraciones, pruebas, workflows y documentación. La carpeta Flor_Arte_Fronted conserva la denominación utilizada por el proyecto.

Estructura de FlorArte

```
Flor_Arte/
  Flor_Arte_Fronted/src/
    App.css
    App.tsx
    assets/
    components/
    config/
    context/
    hooks/
    index.css
    main.tsx
    pages/
    routes/
    services/
    styles/
    types/
    utils/
  Flor_Arte_Backend/src/main/
    java/
    resources/
  Flor_Arte_Backend/src/test/
    java/
  .github/workflows/
    backend-ci.yml
    deploy-production.yml
    docker-ci.yml
    docker-publish.yml
    frontend-ci.yml
    test-aws-connection.yml
  compose.yaml
  DataBase/
  documentacion/
```

Figura T3. Estructura de directorios del proyecto.

Vista elaborada a partir de los nombres de directorios y archivos del repositorio.

El directorio del frontend se denomina Flor_Arte_Fronted; los comandos del proyecto utilizan esa ruta.

Control de versiones

El proyecto utiliza main para la línea principal y develop para la integración. Las ramas feature/* agrupan los cambios de desarrollo. El historial y las ejecuciones de CI relacionan los cambios con sus commits.

Historial reciente, sin autores/datos personales:

Commit	Fecha registrada	Asunto
9205eff	2026-10-19	mensajes gmail
1bc23f3	2026-10-06	reportes y exportaciones
0087541	2026-10-05	modulo de arreglo
727d2e9	2026-10-05	mejoras visuales
232058b	2026-10-04	mejoras implementadas
8dae1e4	2026-09-21	Fase 2
ed41a28	2026-09-21	Documentacion
b07cdb3	2026-09-20	modulo pedidos listo

Archivos ignorados y control de versiones

El control de versiones excluye dependencias instaladas, compilaciones y configuración privada. El lockfile de npm y Maven Wrapper permiten reproducir la instalación y la construcción.

9. Integración continua y entrega de imágenes

9.1. Flujo de integración

El desarrollo se organiza en ramas feature/*, develop y main. GitHub Actions recupera el repositorio y ejecuta los trabajos que corresponden a los cambios. Las verificaciones de frontend, backend y construcción de contenedores son independientes.

Workflow	Archivo	Función
Frontend CI	frontend-ci.yml	Instalación con lockfile, ESLint, TypeScript y build Vite.
Backend CI	backend-ci.yml	Maven, pruebas unitarias e integración con PostgreSQL temporal.
Docker CI	docker-ci.yml	Construcción de imágenes de frontend y backend mediante Buildx.
Publicación GHCR	docker-publish.yml	Construcción y publicación manual de imágenes con etiquetas de commit.

9.2. Ejecuciones de GitHub Actions

Los registros de integración incluyen Backend CI #3, commit 723b385, con trabajos de compilación y pruebas de integración PostgreSQL, y Docker CI #1, commit 4e46b59, en develop. Las figuras siguientes conservan los resultados de esas ejecuciones.

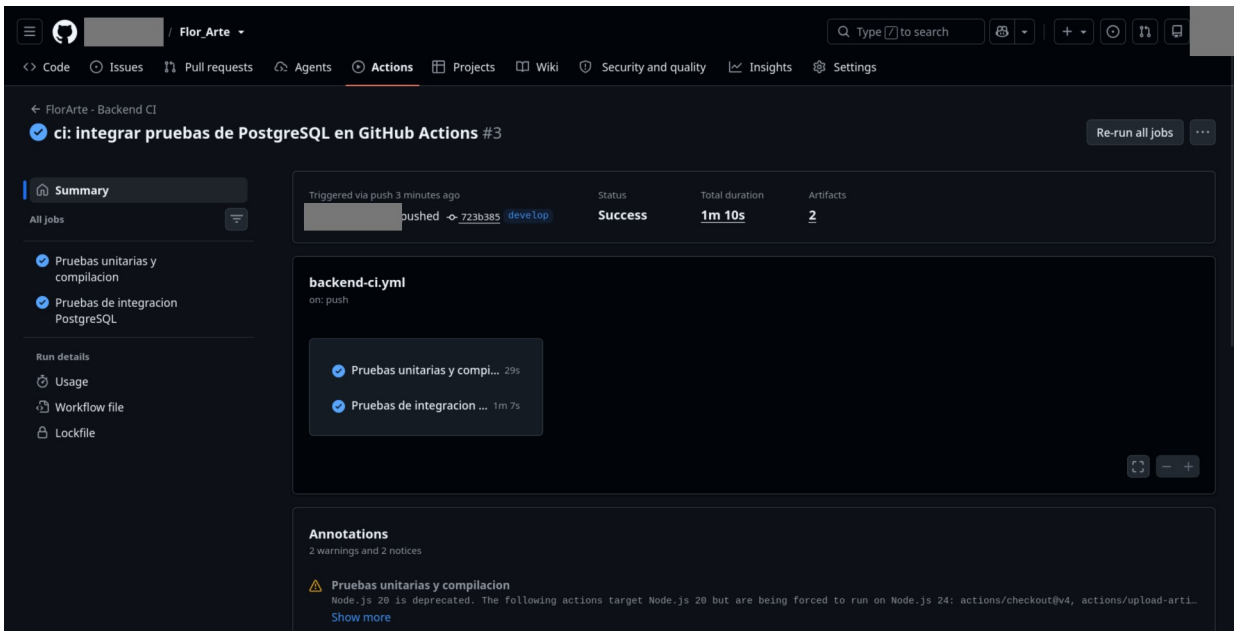


Figura T4. Backend CI #3: trabajos correctos y artefactos de pruebas.

GitHub Actions; captura del registro de desarrollo del proyecto, octubre de 2026. Las cuentas y avatares se han ocultado.

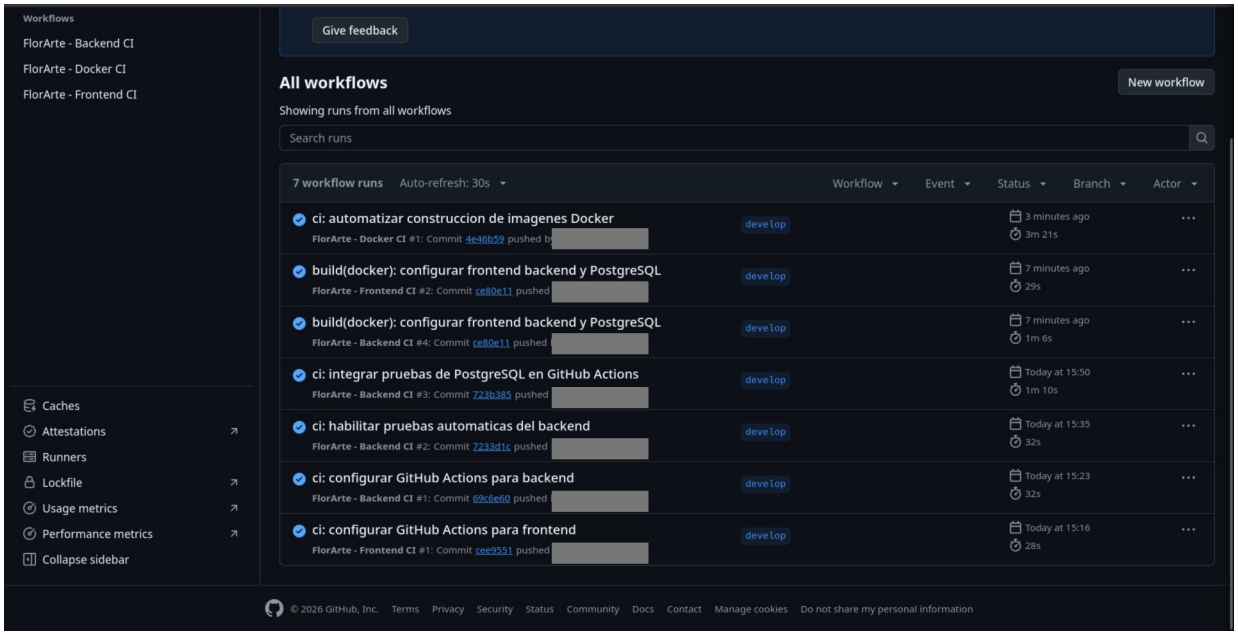


Figura T5. Ejecuciones de frontend, backend y construcción Docker.

GitHub Actions; registro de desarrollo en develop. La construcción de imágenes es independiente de su publicación y despliegue.

9.3. Registro GHCR y distribución

El workflow docker-publish.yml utiliza GHCR como registro de contenedores. Define imágenes separadas para frontend y backend y etiquetas asociadas al commit. La configuración distingue la construcción de imágenes de su publicación en el registro.

La publicación de imágenes y la actualización de EC2 son procesos separados. Las imágenes permiten identificar la versión del artefacto utilizado por Docker Compose y relacionarla con el código de origen.

10. Despliegue y operación

10.1. Arquitectura de producción

La instalación de FlorArte descrita en el proyecto utiliza AWS EC2 con Ubuntu 24.04, Docker Compose y PostgreSQL 17. Caddy proporciona la entrada pública por HTTPS mediante el dominio florarte-xela.duckdns.org; Nginx sirve el frontend React y el backend Spring Boot atiende la API.

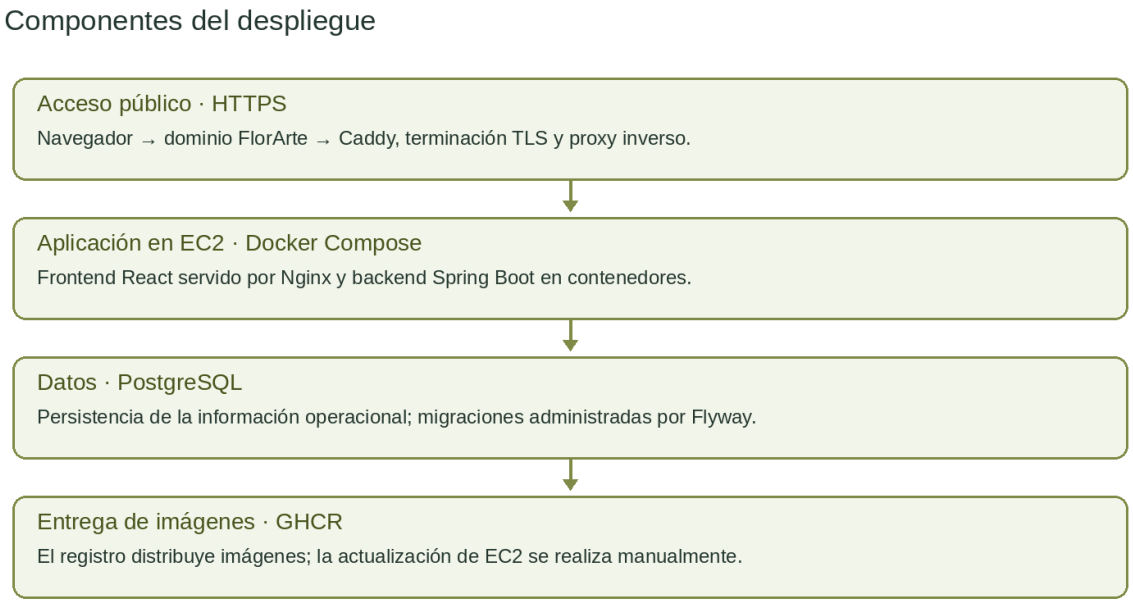


Figura T6. Componentes y recorrido de una solicitud en el despliegue.

Representación funcional de los componentes descritos para la instalación de FlorArte.

10.2. Dominio y HTTPS

El acceso público se realiza mediante https://florarte-xela.duckdns.org. La comprobación del 26 de octubre de 2026 devolvió HTTP 200 y el encabezado de servidor Caddy, con la página principal de FlorArte disponible por HTTPS.



Figura T7. Portada pública de FlorArte servida mediante HTTPS.

Dominio: https://florarte-xela.duckdns.org · 26 de octubre de 2026.

10.3. Contenedores y actualización manual

El Compose del proyecto reúne frontend, backend y PostgreSQL. La base utiliza almacenamiento persistente y una comprobación de salud; el backend se inicia después de que PostgreSQL está disponible y el frontend depende del servicio de aplicación.

Archivo	Función
Flor_Arte_Backend/Dockerfile	Build multietapa Maven/Java 21 y runtime JRE 21, usuario no root; puerto 8080
Flor_Arte_Fronted/Dockerfile	Build Node 24 de React/Vite y servidor Nginx
Flor_Arte_Fronted/nginx.conf	Nginx sirve SPA y reenvía /api/* al backend quitando prefijo /api
compose.yaml	Servicios db, backend y frontend; volumen PostgreSQL; healthcheck; puerto local 127.0.0.1:8081
Configuración de entorno	Parámetros externos de conexión, seguridad y compilación administrados según el entorno de ejecución.
.dockerignore	Excluye paquetes instalados, builds previos y archivos privados del contexto

La actualización de EC2 se realiza manualmente. El operador selecciona la versión de las imágenes, preserva los datos persistentes y comprueba la disponibilidad de la aplicación después del reinicio de los servicios.

El workflow de despliegue define autenticación federada OIDC y ejecución mediante AWS SSM. La autenticación presenta un error al asumir el rol federado; por ello, esta vía no se considera un mecanismo operativo de actualización automática.

10.4. Respaldo y restauración de datos

PostgreSQL permite generar respaldos lógicos con pg_dump y restaurarlos con pg_restore. Para comprobar el procedimiento se respaldó la base local de demostración y se restauró en otra base del mismo entorno aislado. La comparación de personas, usuarios, flores, pedidos y comprobantes coincidió entre origen y destino.

Respaldo y restauración local

Base de demostración con datos ficticios · 26 de octubre de 2026

Registros	Origen	Restauración
Personas	4	4
Usuarios	2	2
Flores	3	3
Pedidos	2	2
Comprobantes	3	3
Historial Flyway	19	19

Resultado: coincidencia en los seis conteos comprobados.

Formato de respaldo: PostgreSQL custom · 91623 bytes

Figura T8. Comparación de conteos tras la restauración local.

La comprobación utiliza exclusivamente registros ficticios y documenta el procedimiento de recuperación de la base demostrativa.

La operación de respaldo debe conservar la fecha, el tamaño y la integridad del archivo. La restauración se comprueba en una base separada antes de utilizar el respaldo como mecanismo de recuperación de una instalación.

10.5. Mantenimiento

El mantenimiento comprende la revisión del espacio disponible, los registros de aplicación, la salud de los contenedores y el certificado HTTPS. Las actualizaciones deben respetar la compatibilidad de las migraciones y contar con un respaldo recuperable.

Las credenciales y claves se administran como parámetros privados. La autorización del backend, las políticas de acceso a las imágenes y los permisos de operación de EC2 forman parte del control de la instalación.